

Sri Lanka Institute of Information Technology



Assignment 01 Report

Data Warehousing and Business Intelligence

IT3021

Campus: SLIIT Kandy Uni

Date: 29/03/2026

	Student Registration Number	Student Name	Student Email
3	[REDACTED]	ABEYKOON A.M.D.S.K	[REDACTED]@my.sliit.lk

Table of Contents

Task 1: Data set selection	4
1.1. Dataset Overview	4
1.2. Business Context.....	4
1.3. Key Attributes in the dataset	5
1.4. Reasons for Dataset Selection	6
1.5. ER Diagram.....	6
1.6. Assumptions for ER Diagram	7
Task 2: Preparation of data sources.....	7
Source 1 – SQL Server Database	7
Source 2 – CSV File.....	8
Source 3 – Excel File	10
Hierarchical planning	10
Task 3: Solution architecture.....	11
1. High-Level Architectural Diagram.....	11
2. Main Components	11
2.1 Data Sources (Excel / CSV / SQL Server)	11
2.2 Staging Area	11
2.3 ETL Layer.....	11
2.4 Data Warehouse.....	12
2.5 BI/ Reporting Layer.....	12
Task 4: Data warehouse design & development	12
4.1 Dimensional Model Overview	12
4.2 Fact Table (Fact_Transaction)	12
4.3 Dimensional Tables	13
4.4 Grain of the Fact Table	14
4.5 Slowly Changing Dimension (SCD)	14
4.6 Design Assumptions	14
4.7 Dimensional Modelling.....	15
4.8 Relational Diagram (Star Schema).....	17
Task 5: ETL Development	18
5.1 Overview	18
5.2 SQL Server Connection.....	18
5.3 ETL Packages and Control Flow.....	19
Package 1 : Load_Dim.dstx.....	19

Package 2: Load_Fact_Transaction.dtsx	26
5.4 Execution Order	28
5.5 Validation of ETL results.....	29
Task 6: ETL development – Accumulating fact tables.....	33
1. Modify Existing Fact Transaction	33
2. Add derived column for accm_txn_create_time to get the date	33
3.Initial Load Logic	34
4.Create Update Dataset (txn_updates.txt).....	35
5.Update ETL Package (Update_Fact_Transaction).....	35
6.Error Handling.....	37
7.Validation Queries	39

Task 1: Data set selection

1.1. Dataset Overview

The dataset of Perishable Goods Management ensure that perishable goods are stored, transported and sold efficiently before expiration while minimizing waste and maximizing profitability. This involves continuous track of product conditions, demand patterns and inventory levels across supply chain. The dataset follows OLTP characteristics like high granularity, transactional-level records, and operational data suitable for real time processing. The dataset supports dimensional modelling by separating measurable facts and descriptive attributes making it suitable for star-schema design.

The selected dataset represents **Perishable Goods Management** records that used in a retail supply chain environment. The dataset captures Transactional and Operational data related to shelf life, spoilage risk and revenue optimization for perishable goods. This dataset contained approximately **100,000 records** of perishable product transactions across 50 grocery stores, 5 US regions and 20 suppliers over a period of **more than one year** (2023-2024) containing **42 features** covering product details, storage conditions, demand patterns, spoilage risk, markdown strategies, and financial outcomes **across 10 product categories** like Dairy, Meat, Seafood, Produce, Bakery, Deli, Frozen Meals, Beverages, Pharmaceuticals, and Ready-to-Eat items. Each record represents a **single product transaction instance** making it suitable as an OLTP dataset.

The dataset expected to identify high spoilage risk of products, optimize price and discount strategies, shelf management, improve supplier performance and forecast demand. By implementing OLTP system enables businesses to reduce waste, improve product quality and enhance overall supply chain efficiency.

Dataset Link - <https://www.kaggle.com/datasets/likithagedipudi/perishable-goods-management>

1.2. Business Context

This dataset can be used to following real-world scenarios where business need to,

1. **Spoilage Prediction** - Predict which products will spoil based on storage/handling conditions
2. **Waste Reduction** - Forecast waste rates to optimize inventory ordering
3. **Demand Forecasting** - Predict daily demand using temporal and categorical features
4. **Markdown Optimization** - Learn optimal discount strategies to maximize profit while minimizing waste
5. **Inventory Management** - Identify stockout risk and overstock patterns

This dataset is useful for Data Warehousing and Business Intelligence that it combines sales, logistics and quality control data.

1.3. Key Attributes in the dataset

2. Product Details

- product_id
- product_name
- category

3. Store and Location Details

- store_id
- region

4. Supplier Details

- supplier_id
- supplier_score

5. Time details

- transaction_date
- expiration_date
- day_of_week
- month
- is_weekend

6. Inventory and Shelf-Life Data

- shelf_life_days
- days_remaining_at_purchase
- days_until_expiry
- initial_quantity

7. Storage and Handling Conditions

- storage_temp
- temp_deviation
- temp_abuse_events
- distribution_hours
- handling_score
- packaging_score

8. Sales and Financial Metrics

- base_price
- cost_price
- selling_price
- waste_cost
- units_sold
- revenue
- profit
- profit_margin_pct

9. Waste and Quality Metrics

- units_wasted
- waste_pct
- spoilage_risk
- spoilage_sensitivity
- was_spoiled
- quality_grade

10. Demand and Promotions

- daily_demand
- demand_variability
- markdown_applied
- discount_pct
- is_promoted

1.4. Reasons for Dataset Selection

- The dataset is an OLTP dataset that contains transaction-level data, not pre-aggregated and suitable for transformation into a data warehouse.
- Supports advanced BI analysis with waste analysis, Track profit of goods and services, supplier performance evaluation and demand forecasting.
- This dataset support data warehouse design containing multiple entities (Product, Store, Suppliers, Time) and enable creation of fact and dimension tables.
- Suitable for ETL process that includes raw and derived attributes that allows data cleaning, transformation, aggregation.

1.5. ER Diagram

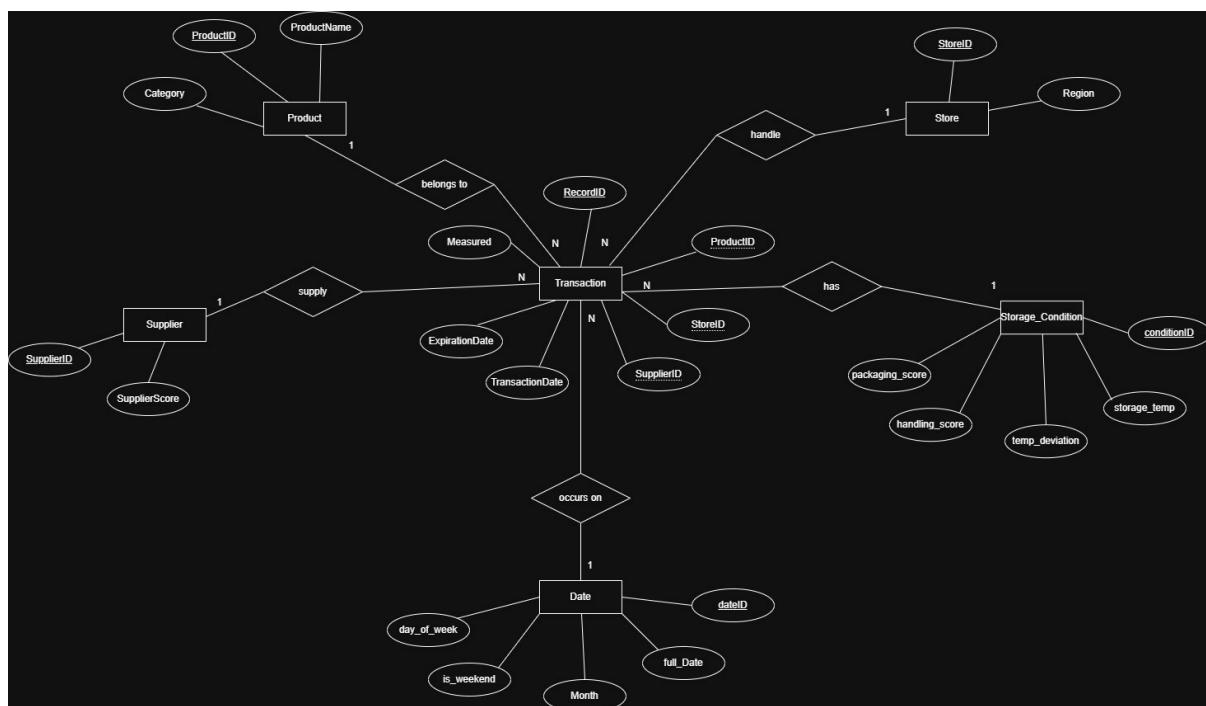


Figure 1 ER Diagram

1.6. Assumptions for ER Diagram

- Each record represents a single product transaction.
- Each product belongs to one category and can appear in many transactions.
- A supplier can supply many transactions, and each transaction has one supplier.
- Each transaction occurs on one date and, can occur many transactions per day.
- Each transaction has one storage condition and storage conditions are grouped.
- Transaction entity acts as the central entity by connecting all other entities.

Task 2: Preparation of data sources

The original dataset was transformed into multiple data sources with different formats (excel, csv, SQL Server Database) to support the ETL process and warehouse design.

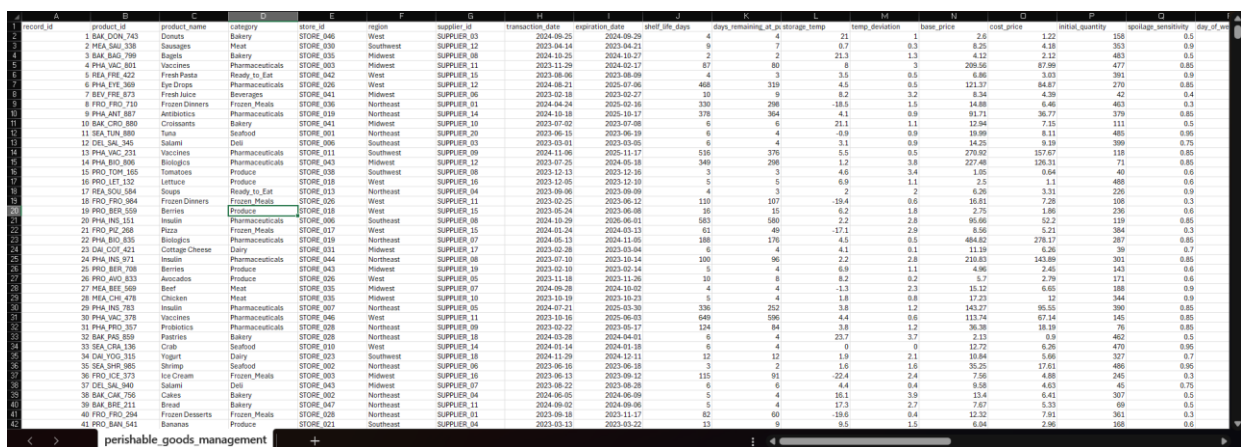


Figure 2: Original Dataset

Source 1 – SQL Server Database

A relational database in SQL Server was used to store **Transaction Data** that represents the main transactional (fact) data and is used for loading the fact in the data warehouse.

- Record ID
- Product ID
- Store ID
- Supplier ID
- Transaction Date
- Expiration Date
- Units Sold
- Units Wasted
- Revenue
- Profit
- Waste Cost
- Base price
- Cost price
- Selling price

Steps of Import Data into SSMS

1. Create a database named PerishableDB.
2. Import data using import flat file functionality for transaction_data.csv into SSMS as a table named 'dbo.TransactionData'.
3. Confirm that the data inserted successfully to the dbo.TransactionData table.

SQLQuery3....\ACER (96))* X new_updates ...A\ACER (82))

```

1
2
3 select * from dbo.TransactionData
  
```

110 % No issues found

Results Messages

record_id	product_id	store_id	supplier_id	transaction_date	expiration_date	units_sold	units_wasted	revenue	waste_cost	profit	base_price	cost_price	selling_price
1	BAK_DON_743	STORE_046	SUPPLIER_03	2024-09-25	2024-09-29	138	20	358.8	24.4	166.04	2.6	1.22	2.6
2	MEA_SAU_338	STORE_030	SUPPLIER_12	2023-04-14	2023-04-21	251	102	2070.75	426.36	595.21	8.25	4.18	8.25
3	BAK_BAG_799	STORE_035	SUPPLIER_08	2024-10-25	2024-10-27	483	0	618.24	0	-405.72	4.12	2.12	1.28
4	PHA_VAC_801	STORE_003	SUPPLIER_11	2023-11-29	2024-02-17	477	0	99960.12	0	57988.89	209.56	87.99	209.56
5	REA_FRE_422	STORE_042	SUPPLIER_15	2023-08-06	2023-08-09	391	0	2064.48	0	879.75	6.86	3.03	5.28
6	PHA_EYE_369	STORE_026	SUPPLIER_12	2024-08-21	2025-07-06	0	270	0	22914.9	-22914.9	121.37	84.87	121.37
7	BEV_FRE_873	STORE_041	SUPPLIER_06	2023-02-18	2023-02-27	41	1	341.94	4.39	157.56	8.34	4.39	8.34
8	FRO_FRO_710	STORE_036	SUPPLIER_01	2024-04-24	2025-02-16	0	463	0	2990.98	-2990.98	14.88	6.46	14.88
9	PHA_ANT_887	STORE_019	SUPPLIER_14	2024-10-18	2025-10-17	379	0	34758.09	0	20822.26	91.71	36.77	91.71
10	BAK_CRO_880	STORE_041	SUPPLIER_10	2023-07-02	2023-07-08	111	0	1436.34	0	642.69	12.94	7.15	12.94
11	SEA_TUN_880	STORE_001	SUPPLIER_20	2023-06-15	2023-06-19	257	228	5137.43	1849.08	1204.08	19.99	8.11	19.99
12	DEL_SAL_345	STORE_006	SUPPLIER_03	2023-03-01	2023-03-05	279	120	3975.75	1102.8	308.94	14.25	9.19	14.25
13	PHA_VAC_231	STORE_011	SUPPLIER_09	2024-11-06	2025-11-17	118	0	24935.76	0	6330.7	270.92	157.67	211.32
14	PHA_BIO_806	STORE_043	SUPPLIER_12	2023-07-25	2024-05-18	0	71	0	8968.01	-8968.01	227.48	126.31	227.48
15	PRO TOM 165	STORE 038	SUPPLIER 08	2023-12-13	2023-12-16	40	0	34	0	8.4	1.05	0.64	0.85

Figure 3: SQL Server Transaction Data Table

Source 2 – CSV File

A CSV file named **product_supplier.csv** was created to store product and supplier related information and used to populate the Product and Supplier dimensions.

- Product ID
- Product Name
- Category
- Supplier ID
- Supplier Score
- Spoilage Sensitivity

	A	B	C	D	E	F
1	product_id	product_name	category	supplier_id	spoilage_sensitivity	supplier_score
2	BAK_DON_743	Donuts	Bakery	SUPPLIER_03	0.5	9
3	MEA_SAU_338	Sausages	Meat	SUPPLIER_12	0.9	6
4	BAK_BAG_799	Bagels	Bakery	SUPPLIER_08	0.5	9
5	PHA_VAC_801	Vaccines	Pharmaceuticals	SUPPLIER_11	0.85	6
6	REA_FRE_422	Fresh Pasta	Ready_to_Eat	SUPPLIER_15	0.9	6
7	PHA_EYE_369	Eye Drops	Pharmaceuticals	SUPPLIER_12	0.85	6
8	BEV_FRE_873	Fresh Juice	Beverages	SUPPLIER_06	0.4	8
9	FRO_FRO_710	Frozen Dinners	Frozen_Meals	SUPPLIER_01	0.3	7
10	PHA_ANT_887	Antibiotics	Pharmaceuticals	SUPPLIER_14	0.85	6
11	BAK_CRO_880	Croissants	Bakery	SUPPLIER_10	0.5	8
12	SEA_TUN_880	Tuna	Seafood	SUPPLIER_20	0.95	9
13	DEL_SAL_345	Salami	Deli	SUPPLIER_03	0.75	9
14	PHA_VAC_231	Vaccines	Pharmaceuticals	SUPPLIER_09	0.85	8
15	PHA_BIO_806	Biologics	Pharmaceuticals	SUPPLIER_12	0.85	6
16	PRO TOM 165	Tomatoes	Produce	SUPPLIER_08	0.6	9
17	PRO LET 132	Lettuce	Produce	SUPPLIER_16	0.6	10
18	REA SOU 584	Soups	Ready_to_Eat	SUPPLIER_04	0.9	9
19	FRO_FRO_984	Frozen Dinners	Frozen_Meals	SUPPLIER_11	0.3	6
20	PRO BER 559	Berries	Produce	SUPPLIER_15	0.6	6
21	PHA INS 151	Insulin	Pharmaceuticals	SUPPLIER_08	0.85	9
22	FRO PIZ 268	Pizza	Frozen_Meals	SUPPLIER_15	0.3	6
23	PHA BIO 835	Biologics	Pharmaceuticals	SUPPLIER_07	0.85	9
24	DAL COT 421	Cottage Cheese	Dairy	SUPPLIER_17	0.7	9
25	PHA INS 971	Insulin	Pharmaceuticals	SUPPLIER_08	0.85	9
26	PRO BER 708	Berries	Produce	SUPPLIER_19	0.6	10

Figure 4: product_supplier.csv

A CSV file named **storage_conditions.csv** was created to store storage and condition handling related information and used to support advanced analysis to product spoilage and quality.

- Storage Temperature
- Temp Deviation
- Handling Score
- Packaging Score

	A	B	C	D	
1	storage_temp	temp_deviation	handling_score	packaging_score	
2	21	1	9	10	
3	0.7	0.3	8	8	
4	21.3	1.3	4	8	
5	8	3	5	9	
6	3.5	0.5	5	6	
7	4.5	0.5	10	8	
8	8.2	3.2	8	9	
9	-18.5	1.5	5	7	
10	4.1	0.9	5	7	
11	21.1	1.1	4	8	
12	-0.9	0.9	6	6	
13	3.1	0.9	7	5	
14	5.5	0.5	4	5	
15	1.2	3.8	8	5	
16	4.6	3.4	5	6	
17	6.9	1.1	6	6	
18	2	2	5	6	
19	-19.4	0.6	5	7	
20	6.2	1.8	7	6	
21	2.2	2.8	6	9	
22	-17.1	2.9	5	6	
23	4.5	0.5	10	6	
24	4.1	0.1	7	5	
25	2.2	2.8	10	7	
26	6.9	1.1	6	6	

Figure 5: storage_conditions.csv

A CSV file named **date.csv** was created to store date related information and used to support time and period analysis for sales patterns.

- full_date (transaction_date)
- date_key
- day of week
- is_weekend
- month
- day
- year
- quarter
- week_of_year

	A	B	C	D	E	F	G	H	I	
1	full_date	date_key	day_of_week	is_weekend	month	day	year	quarter	week_of_year	
2	2024-09-25	20240925	2	0	9	25	2024	3	39	
3	2023-04-14	20230414	4	0	4	14	2023	2	15	
4	2024-10-25	20241025	4	0	10	25	2024	4	43	
5	2023-11-29	20231129	2	0	11	29	2023	4	48	
6	2023-08-06	20230806	6	1	8	6	2023	3	32	
7	2024-08-21	20240821	2	0	8	21	2024	3	34	
8	2023-02-18	20230218	5	1	2	18	2023	1	7	
9	2024-04-24	20240424	2	0	4	24	2024	2	17	
10	2024-10-18	20241018	4	0	10	18	2024	4	42	
11	2023-07-02	20230702	6	1	7	2	2023	3	27	
12	2023-06-15	20230615	3	0	6	15	2023	2	24	
13	2023-03-01	20230301	2	0	3	1	2023	1	9	
14	2024-11-06	20241106	2	0	11	6	2024	4	45	

Figure 6: date.csv

Source 3 – Excel File

An Excel file named **store_data.xlsx** was created to store store-related information enabling the creation of a Store dimension with hierarchical structure (Region --> Store) and contain 50 unique store ids.

- Store ID
- Region

	A	B	C
1	store_id	region	
26	STORE_047	Northeast	
27	STORE_021	Southeast	
28	STORE_009	Northeast	
29	STORE_020	West	
30	STORE_033	Southeast	
31	STORE_045	Southwest	
32	STORE_015	Southeast	
33	STORE_050	West	
34	STORE_022	West	
35	STORE_025	Southwest	
36	STORE_012	Northeast	
37	STORE_049	Northeast	
38	STORE_037	Northeast	
39	STORE_005	Southeast	
40	STORE_039	Northeast	
41	STORE_016	Southeast	
42	STORE_014	Northeast	
43	STORE_008	West	
44	STORE_034	Southeast	
45	STORE_024	Southeast	
46	STORE_040	Midwest	
47	STORE_048	Southwest	
48	STORE_004	Southeast	
49	STORE_027	Midwest	
50	STORE_032	Midwest	
51	STORE_029	Southeast	

Figure 7: store_data.xlsx

Transaction related attributes like sales, revenue, quantity are stored in SQL Server, while descriptive attributes such as product, supplier, store and date details are separated into CSV and Excel files to support dimension table creation in the data warehouse.

During the data extraction into different data sources the **duplicated records were removed** from the **product_supplier.csv** and **store_data.xlsx** to ensure uniqueness of dimensions attributes and to avoid redundancy during dimensions loading in ETL process.

The original dataset was successfully divided into multiple data sources including CSV, Excel and SQL Server Database formats enables handling fact and dimension data, allowing effective implementation of ELT processes and data warehouse modelling.

Hierarchical planning

Data enrichment was performed by deriving additional attributes like date components (year,month,quarter) and supplier performance measures. Hierarchies such as Region – Store – Year – Month – Day were defined to support drill-down analysis in the data warehouse.

Task 3: Solution architecture

1. High-Level Architectural Diagram



Figure 8: high-level architectural diagram

2. Main Components

The processed data warehouse architecture follows a layered approach including of data sources, staging area, ETL layer, data warehouse (DW) and reporting layer. Data is extracted from multiple sources, transformed using SSIS and loaded into a centralized data warehouse for analysis.

The data flows from heterogeneous sources into the staging area where it is cleansed and transformed using SSIS, then loaded into the data warehouse and finally consumed by BI tools for reporting and analysis.

2.1 Data Sources (Excel / CSV / SQL Server)

The system uses multiple data sources such as CSV files, Excel files and SQL Server database where data is stored as raw unprocessed data.

- dbo.TransactionData – SQL Server Transactional Data Table (fact)
- product_supplier.csv – Product and Supplier dimension
- store_data.xlsx – store dimension
- storage_conditions.csv – storage /handling dimension
- date.csv – date dimension

2.2 Staging Area

A temporary area in SQL Server where the store raw data extracted from different sources. This allows data cleaning and data transformation before loading into the data warehouse. (ex: stg_transaction, stg_product, stg_store, stg_storage, stg_date, stg_supplier)

2.3 ETL Layer

Use SQL Server Integration Services (SSIS) packages to perform ETL operations, extract data from sources, prepare data to be inserted to the target model including data cleansing, type conversions, business logic, integrating (matching and combining), de-coupling, enriching calculations and aggregations. Then load data into warehouse tables. (ex: data conversion, lookup, derived columns)

2.4 Data Warehouse

This is the final structured of database that process architectures and technologies used for collecting and managing data from various sources to support deriving meaningful business insights from the raw data. The data warehouse is designed using a central fact table (fact_transaction), and multiple dimension tables (dim_product, dim_store, dim_supplier, dim_storage, dim_date).

2.5 BI/ Reporting Layer

The data warehouse supports OLAP operations such as slicing, dicing, drill-down and roll-up to enable multidimensional analysis of business data. Data warehouse supports business intelligence tools such as Power BI, tableau and Excel, enabling reporting, visualization and analysis of key metrics like profit, waste, revenue and spoilage trends.

Task 4: Data warehouse design & development

4.1 Dimensional Model Overview

For the perishable Goods dataset, a Star Schema was designed to support analytical processing and reporting by ensuring optimized and efficient dimensional modelling. The schema consists of a central fact table that connected to multiple dimensional tables. The Fact_Transaction table captures the measurable attributes, business events such as sales, waste and profit. Dimensional table provide information of descriptive attributes such as product, store, supplier, storage conditions and date.

The star shema was selected due to simplicity and idea; for business intelligence scenarios where read performance is prioritized over storage efficiency. The denormalized structure reduces the number of joins by makes fast query execution.

4.2 Fact Table (Fact_Transaction)

The central table in the schema that each record represents a single transaction. This table captures the detailed granular data and support drill-down analysis, and fact table stores quantitative measured used for analysis like sales performance, profitability, and waste tracking.

Surrogate Keys (SK) are used to improve join performance and to support Slowly Changing Dimensions, and natural keys to allow efficient joins between fact and dimension tables.

- Units_sold
- Units_wasted
- Revenue
- Profit
- Waste_cost
- Base_price

- Cost_price
- Selling_price
- Expiration_date (descriptive / degenerate attribute)

Expiration_date is chosen as a descriptive attribute in the fact table due to it is directly related to each transaction and it is required for calculating shelf-life, spoilage risk and time-based analysis. It is not treated as a measure but as a supporting attribute for analysis.

Foreign Keys:

- Fact_id (PK)
- Product_key (SK)
- Store_key (SK)
- Supplier_key (SK)
- Storage_key (SK)
- Date_key (SK)

4.3 Dimensional Tables

Dimension tables are created as denormalized dimensions to reduce complexity and improve performance by storing descriptive attributes.

Dim_Product – product related information (SCD Type 2)

- Product_key (PK)
- Product_id
- Product_name
- Category
- Spoilage_sensitivity
- Start_date
- End_date
- Is_current

Dim_Store – store and regional information

- Store_key (PK)
- Store_id
- region

Dim_Supplier – supplier details (SCD Type 2)

- Supplier_key(PK)
- Supplier_id
- Supplier_score
- Start_date
- End_date
- Is_current

Dim_Storage – storage and handling conditions

- Storage_key (PK)
- Storage_temp
- Temp_deviation

- Handling_score
- Packaging_score

Dim_Date – supports time-based information

- Date_key(PK)
- Full_date
- Day
- Month
- Year
- Day_of_week
- Is_weekend
- Quarter
- Week_of_year

4.4 Grain of the Fact Table

One record in dataset represents a single product transaction at a specific store, supplier, storage condition and date.

4.5 Slowly Changing Dimension (SCD)

Among the dimension tables the Dim_Product and Dim_Supplier tables are considered as Slowly Changing Dimensions (SCD Type 2) to preserve historical changes in dimensional attributes. In Dim_Product attributes like (product name or category) may change over time, while in Dim_Supplier, the supplier_score may change due to performance change. Instead of overwriting existing data and a new record is created to preserve historical changes. Each version of the record is tracked using Start_Date, End_Date and Is_Current files. This allows keep tracking of changes over time and support accurate historical analysis.

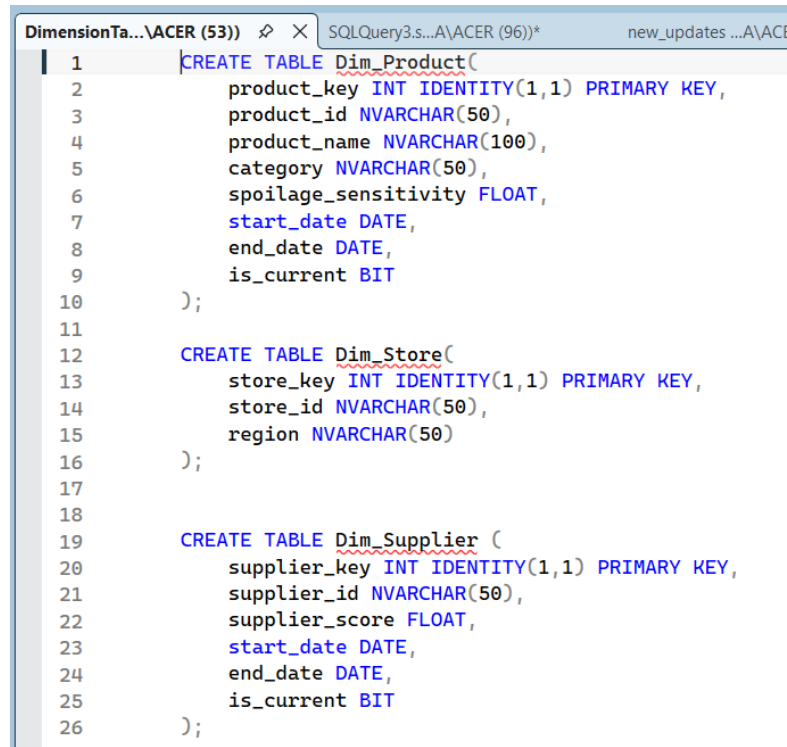
4.6 Design Assumptions

- Each transaction is uniquely identified using primary key and linked to dimensions using surrogate keys.
- Each product belongs to one category.
- Each store belongs to one region.
- Date dimension is pre-generated from transaction dates.
- Storage conditions are grouped into a separate dimension.
- All attributes from the original dataset were not included, as only relevant required analytical attributes were selected and derived attributed (profit_margin_pct, discount_pct) can be calculated during analysis.

4.7 Dimensional Modelling

In SQL Server, 'PerishableDW' database created and the star schema was implemented using,

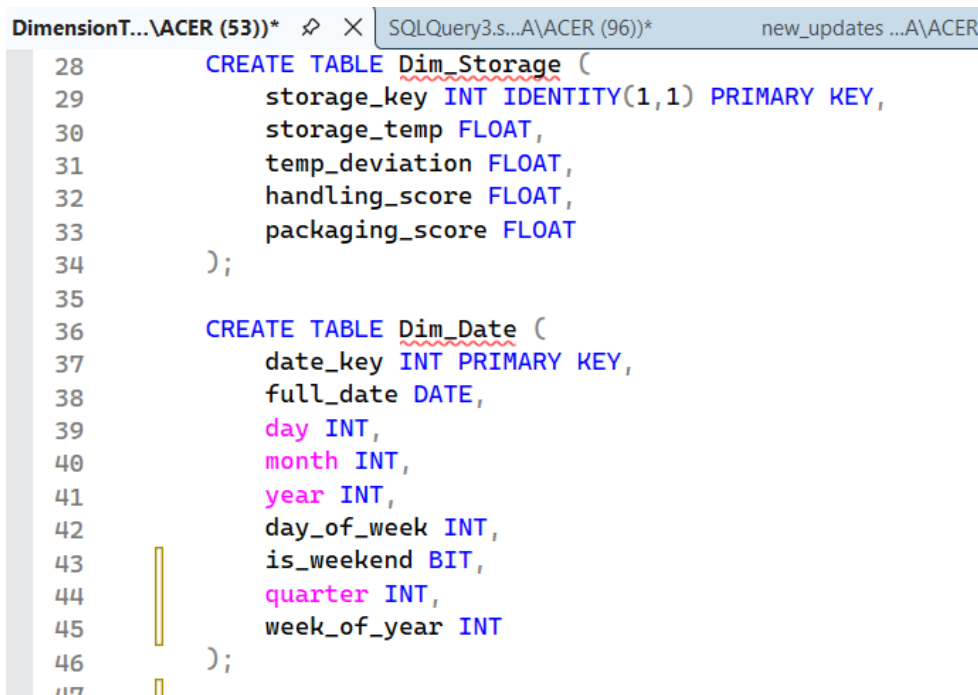
- Dimension tables with surrogate primary keys
- Fact table with foreign key relationships
- Constrains are made to maintain referential integrity.



```

1  CREATE TABLE Dim_Product(
2      product_key INT IDENTITY(1,1) PRIMARY KEY,
3      product_id NVARCHAR(50),
4      product_name NVARCHAR(100),
5      category NVARCHAR(50),
6      spoilage_sensitivity FLOAT,
7      start_date DATE,
8      end_date DATE,
9      is_current BIT
10 );
11
12 CREATE TABLE Dim_Store(
13     store_key INT IDENTITY(1,1) PRIMARY KEY,
14     store_id NVARCHAR(50),
15     region NVARCHAR(50)
16 );
17
18
19 CREATE TABLE Dim_Supplier (
20     supplier_key INT IDENTITY(1,1) PRIMARY KEY,
21     supplier_id NVARCHAR(50),
22     supplier_score FLOAT,
23     start_date DATE,
24     end_date DATE,
25     is_current BIT
26 );
  
```

Figure 9: SQL query for Dim_Product, Dim_Store, Dim_Supplier



```

28 CREATE TABLE Dim_Storage (
29     storage_key INT IDENTITY(1,1) PRIMARY KEY,
30     storage_temp FLOAT,
31     temp_deviation FLOAT,
32     handling_score FLOAT,
33     packaging_score FLOAT
34 );
35
36 CREATE TABLE Dim_Date (
37     date_key INT PRIMARY KEY,
38     full_date DATE,
39     day INT,
40     month INT,
41     year INT,
42     day_of_week INT,
43     is_weekend BIT,
44     quarter INT,
45     week_of_year INT
46 );
  
```

Figure 10: SQL query for Dim_Storage, Dim_Date

DimensionT... \ACER (53))* SQLQuery3.s... A\ACER (96))* new_updates ... A\ACER (82))

```

53
54 CREATE TABLE Fact_Transaction (
55     fact_id INT IDENTITY(1,1) PRIMARY KEY,
56
57     product_key INT,
58     store_key INT,
59     supplier_key INT,
60     date_key INT,
61     storage_key INT,
62
63     units_sold INT,
64     units_wasted INT,
65
66     base_price FLOAT,
67     cost_price FLOAT,
68     selling_price FLOAT,
69
70     revenue FLOAT,
71     profit FLOAT,
72     waste_cost FLOAT,
73
74     expiration_date DATE,
75
76     FOREIGN KEY (product_key) REFERENCES Dim_Product(product_key),
77     FOREIGN KEY (store_key) REFERENCES Dim_Store(store_key),
78     FOREIGN KEY (supplier_key) REFERENCES Dim_Supplier(supplier_key),
79     FOREIGN KEY (date_key) REFERENCES Dim_Date(date_key),
80     FOREIGN KEY (storage_key) REFERENCES Dim_Storage(storage_key)
81 );
82
  
```

Figure 11: SQL query for Fact_Transaction table

DimensionT... \ACER (53))* SQLQuery3.s... A\ACER (96))* new_updates ... A\ACER (82))

```

83
84 ALTER TABLE Dim_Product
85 ADD CONSTRAINT DF_Product_start DEFAULT GETDATE() FOR start_date;
86
87 ALTER TABLE Dim_Product
88 ADD CONSTRAINT DF_Product_iscurrent DEFAULT 1 FOR is_current;
89
90 ALTER TABLE Dim_Supplier
91 ADD CONSTRAINT DF_Supplier_start DEFAULT GETDATE() FOR start_date;
92
93 ALTER TABLE Dim_Supplier
94 ADD CONSTRAINT DF_Supplier_iscurrent DEFAULT 1 FOR is_current;
95
  
```

Figure 12: constraints for SCD

4.8 Relational Diagram (Star Schema)

A relational diagram was created using SQL Server Management Studio (SSMS) to visualize the star schema design. The diagram consists of central fact table (Fact_Transaction) connected to multiple dimension tables including Dim_Product, Dim_Supplier, Dim_Store, Dim_Storage, and Dim_Date through surrogate key relationships.

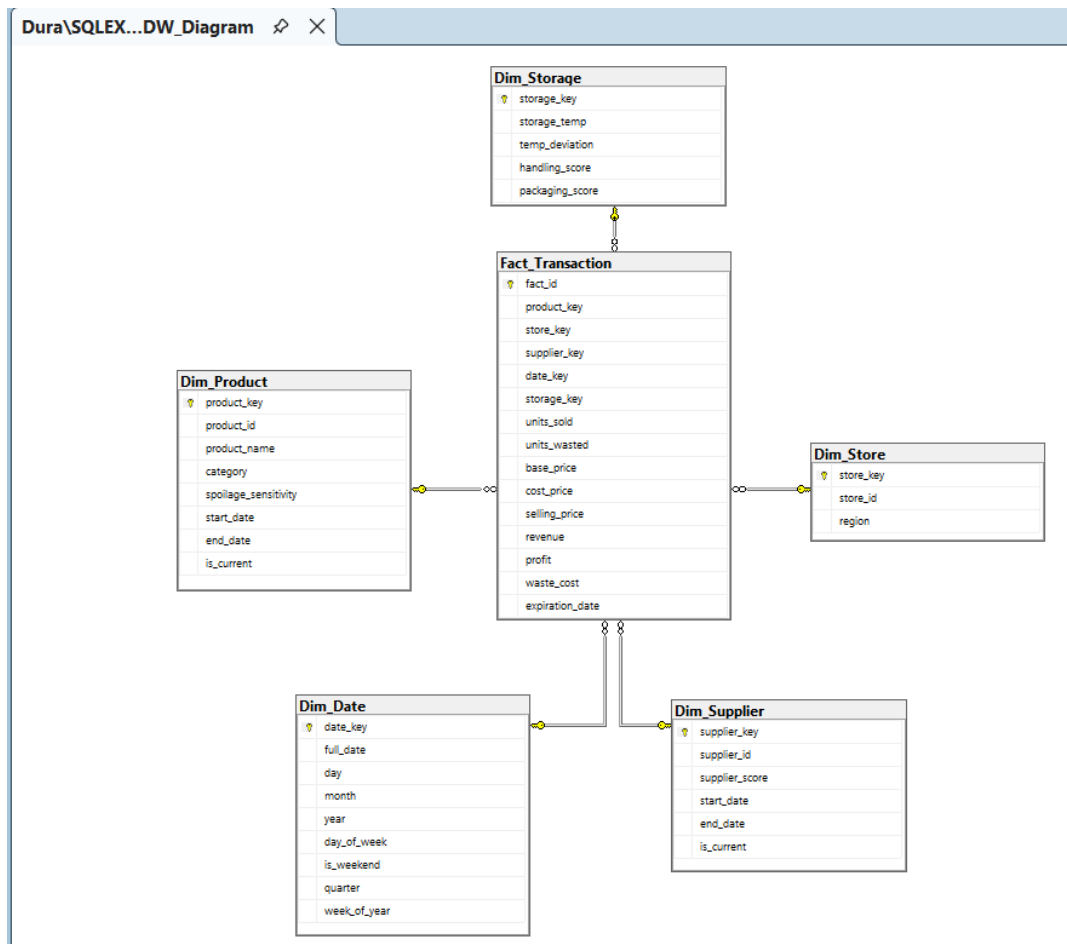


Figure 13: Relational Diagram

Task 5: ETL Development

5.1 Overview

ETL Process was implemented using SQL Server Integration Services (SSIS) packages to extract data from multiple sources including CSV, Excel, and SQL Server into a centralized Data Warehouse. Dimension tables were loaded first before the fact table to ensure proper surrogate key mapping. Lookup transformations, data conversions, derived columns, error handling and slowly changing dimension (SCD Type 2) were used in the process to map natural keys to surrogate keys. Data validation was performed after loading, by using SQL queries.

1.Extract Phase – extract data from multiple sources including CSV files,Excel files, and SQL Server tables using SSIS.

2. Transform Phase – Data Cleaning and transformation done using SSIS transformations such as data conversion, derived columns, sort, lookup, SCD transformations.

3. Load Phase – Transformed data is first loaded into staging tables, then load into dimension tables followed by the fact table.

5.2 SQL Server Connection

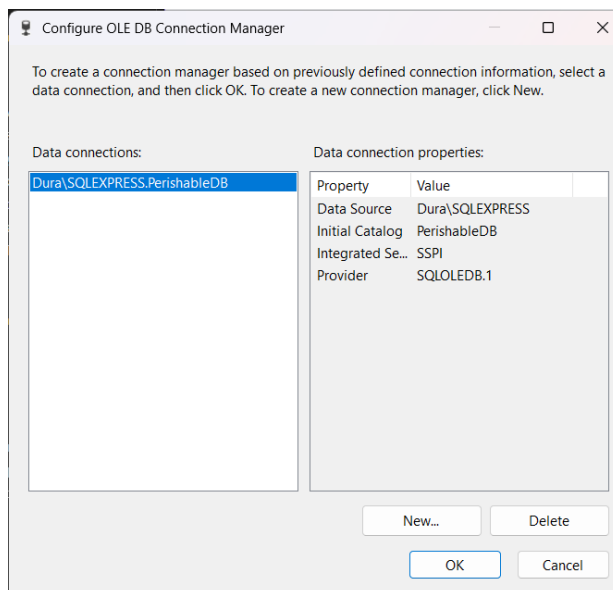


Figure 14:configure OLE DB connection for PerishableDB

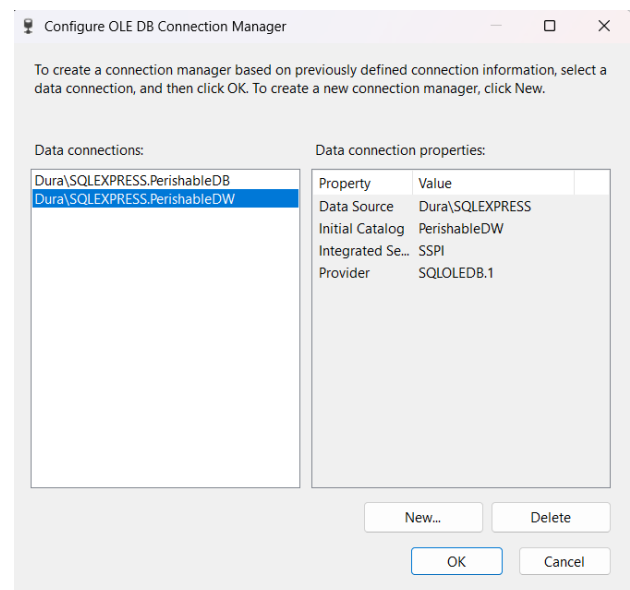


Figure 15: OLE DB connection for PerishableDW

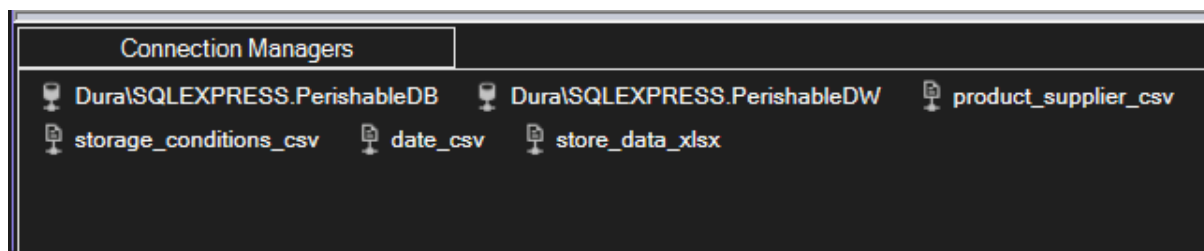


Figure 16:Connection Managers: Data Extraction from multiple sources

5.3 ETL Packages and Control Flow

Package 1 : Load_Dim.dstx

Load all dimension tables and control flow was designed to both staging and dimension loading with a single workflow. Each staging tables are truncated before load new data. Steps in Package 1 are follows,

1.Data extraction from multiple sources

- Csv files (date_csv, prdocut_supplier.csv, storage_conditions.csv)
- Excel file(storage_data.xlsx)

2.Load data into staging tables

- stg_product_supplier
- stg_store
- stg_storage
- stg_date
- stg_transaction

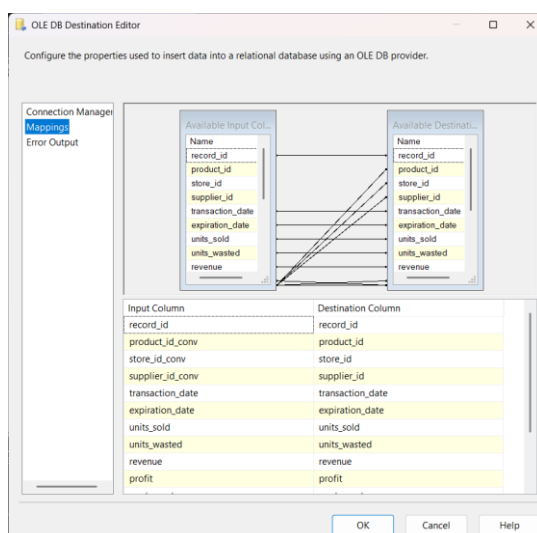


Figure 17:stg_transaction mapping

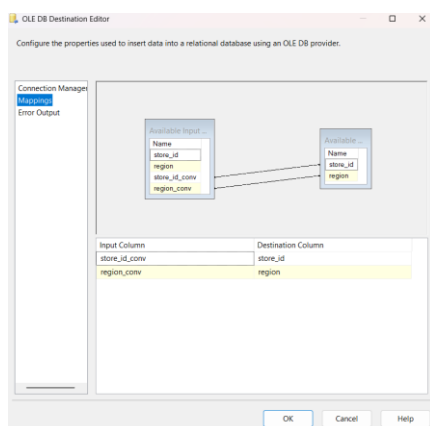


Figure 19: stg_store mapping

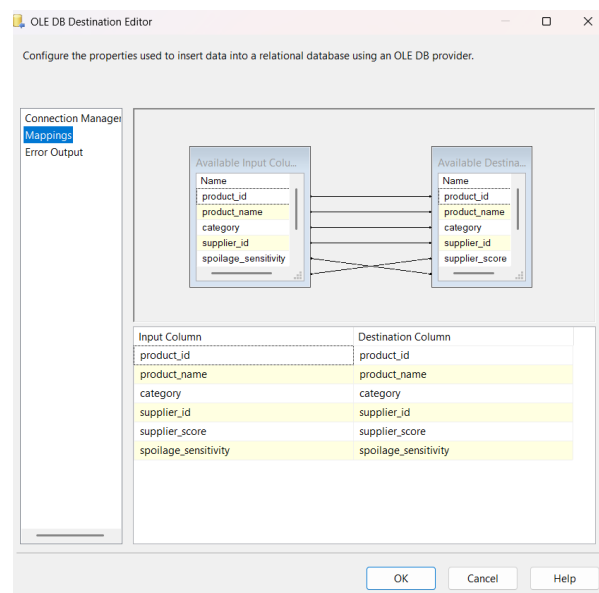


Figure 18: stg_product_supplier mapping

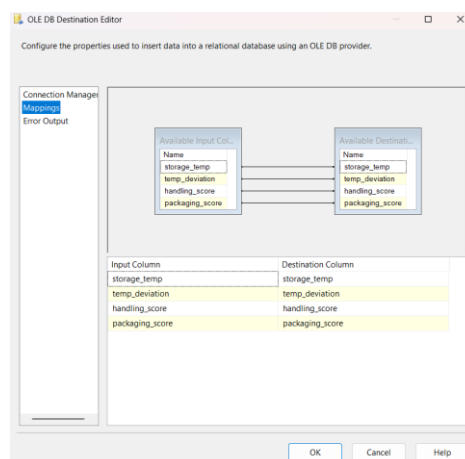


Figure 20: stg_storage mapping

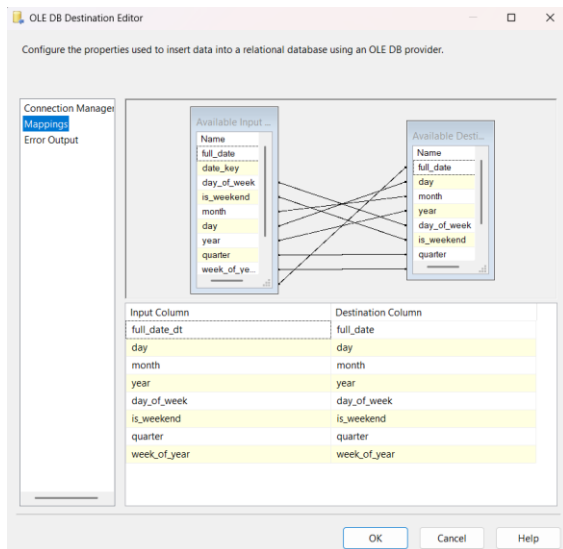


Figure 21: stg_date mapping

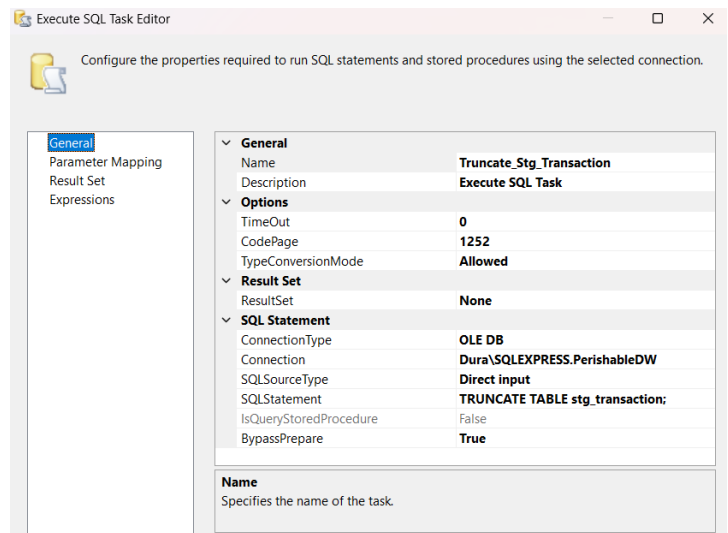


Figure 22: Truncate Staging Tables

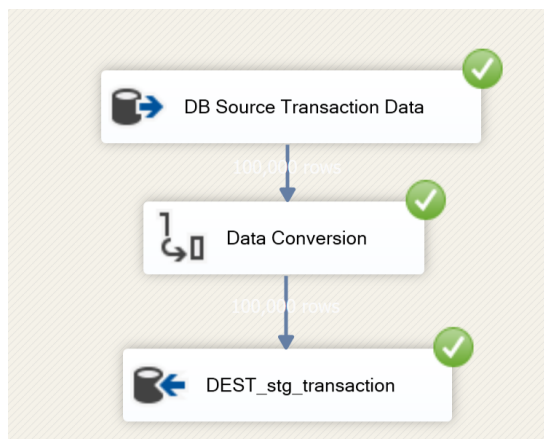


Figure 23: Data Flow of Load_stg_transaction

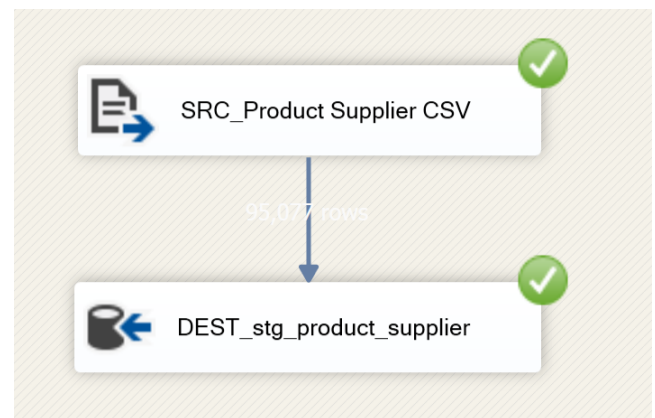


Figure 24: Data Flow of Load_stg_product_supplier

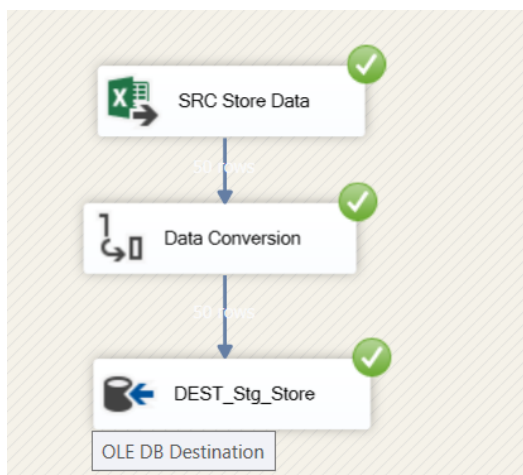


Figure 25: Data Flow of Load_stg_store

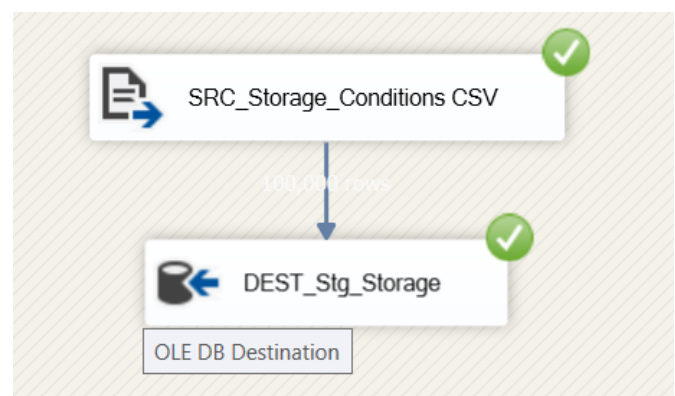


Figure 26: Data Flow of Load_stg_storage_conditions

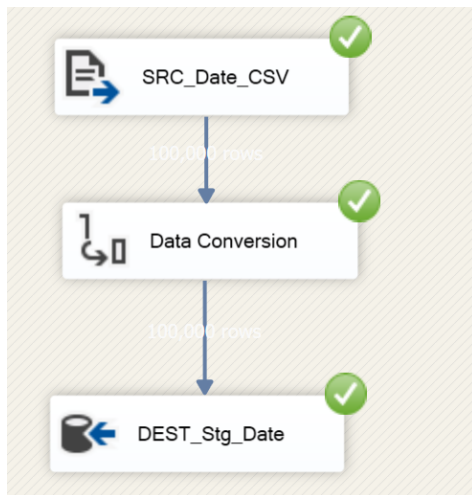


Figure 17: Data Flow of Load_stg_date

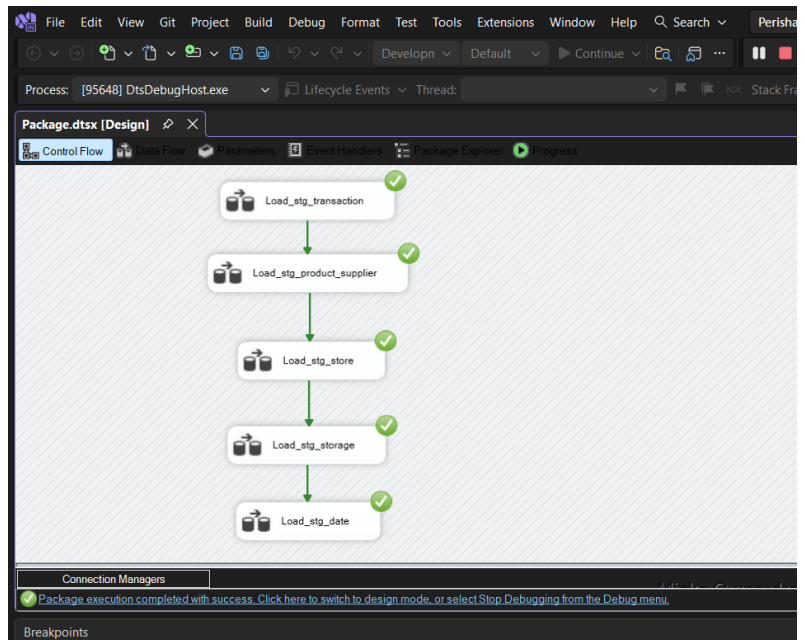


Figure 18: Control Flow of staging tables loading sequence

3. Applying transformations

- Data conversion – ensure all data types are matched between source and destination in data warehouse schema
- Derived columns – create new fields (calculated attributes) like date_key, quarter.
- Sort Transformation - for remove duplicate records in Dim_Date
- Slowly Changing Dimensions (SCD Type 2) – track historical changes in Dim_Product and Dim_Supplier.
- Lookup Transformation – maps natural keys to surrogate keys
- Truncate data – remove all staging data before each load to ensure fresh data processing and to avoid residual data from previous runs.

Derived Column Transformation Editor

Specify the expressions used to create new column values, and indicate whether the values update existing columns or populate new columns.

Derived Column Name	Derived Column	Expression	Data Type
date_key	<add as new column>	(DT_J4)(YEAR(full_date_dt) * 10000 + MONTH(full_date_dt))	four-byte signed integer
day	<add as new column>	(DT_J4)DAY(full_date_dt)	four-byte signed integer
month	<add as new column>	(DT_J4)MONTH(full_date_dt)	four-byte signed integer
year	<add as new column>	(DT_J4)YEAR(full_date_dt)	four-byte signed integer
day_of_week	<add as new column>	(DATEPART("dw", full_date_dt))	four-byte signed integer
is_weekend	<add as new column>	(DATEPART("dw", full_date_dt) = 1 DATEPART("dw", full_date_dt) = 7)	four-byte signed integer
quarter	<add as new column>	(DT_J4)(DATEPART("q", full_date_dt))	four-byte signed integer
week_of_year	<add as new column>	(DT_J4)(DATEPART("wk", full_date_dt))	four-byte signed integer

Figure 29: Derived Columns for Dim_Date

Data Conversion Transformation Editor

Configure the properties used to convert the data type of an input column to a different data type. Depending on the data type to which the column is converted, set the length, precision, scale, and code page of the column.

Input Column	Output Alias	Data Type	Length	Precision	Scale	Code
product_id	product_id_conv	Unicode string [DT_WSTR]	50			
category	category_conv	Unicode string [DT_WSTR]	50			
product_name	product_name_conv	Unicode string [DT_WSTR]	100			

Figure 30: Data Conversion for Dim_Product

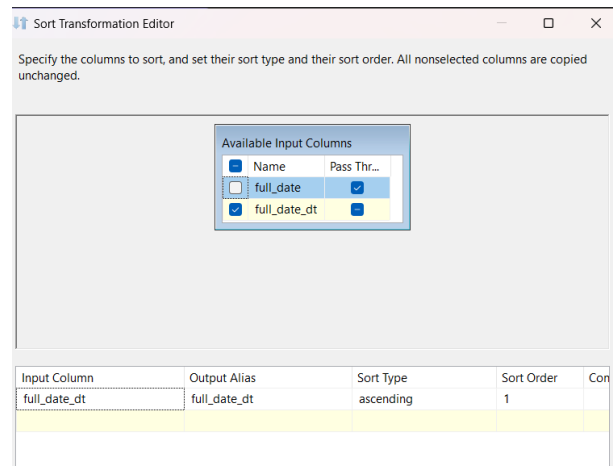


Figure 19: Sort in Dim_Date for full_date

4.Loading into dimension tables steps

1.Dim_Product (SCD Type 2)

The Dim_Product table stores descriptive information about products, and it is used to analyse product performance, category trends and spoilage behavior. This dimension implemented as an SCD Type 2 to track historical changes in product attributes. When a product's details change, a new record inserted while previous records using start_date, end_date, is_current. The SCD transformation compares incoming records with existing dimension records and inserts new rows when changes are detected, while updating the previous record's end_date and is_current flag.

- Data Source – product_supplier.csv
- Data Conversion – convert product_id, product_name, category into DT_WSTR data type.
- SCD Transformation – Business Key: product_id / Type1: category / Type2: product_name
- Destination – Dim_Product

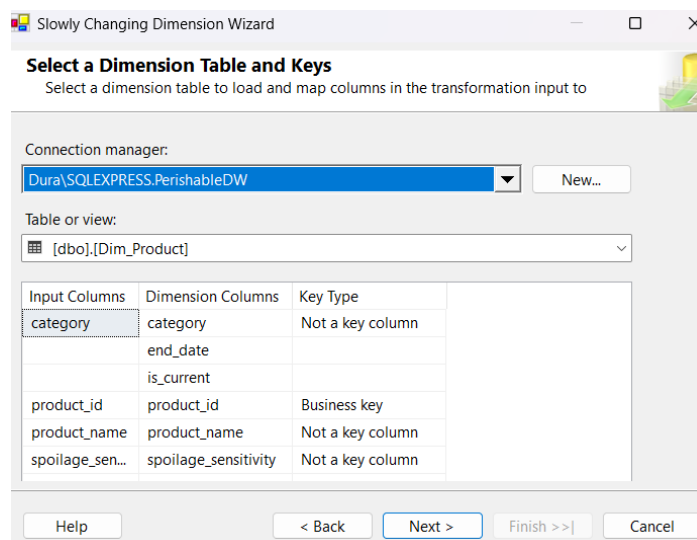


Figure 20: SCD for Dim_Product

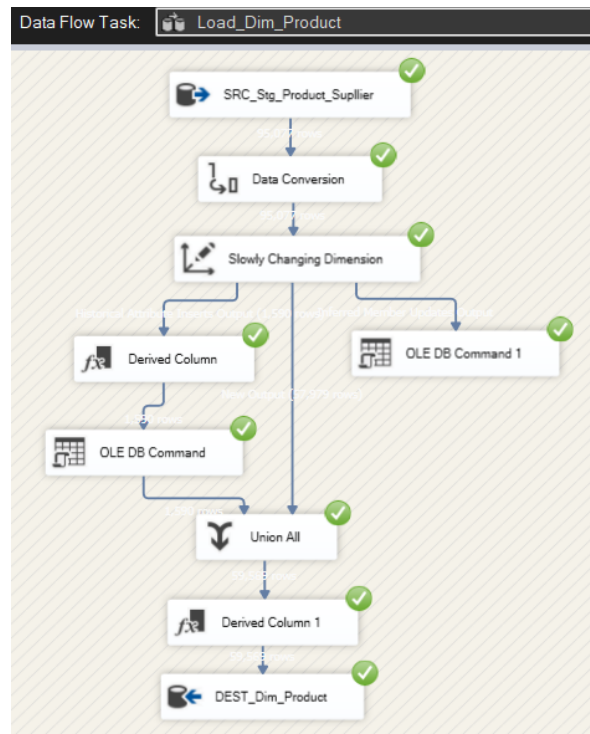


Figure 21:Data Flow of Dim_Product

2. Dim_Store

This Dim_Store table contains information about store locations and regions and it supports regional sales and performance analysis.

- Data Source – store_data.xlsx
- Data Conversion – convert store_id and region to DT_STR (string) type.
- Destination – Dim_Store

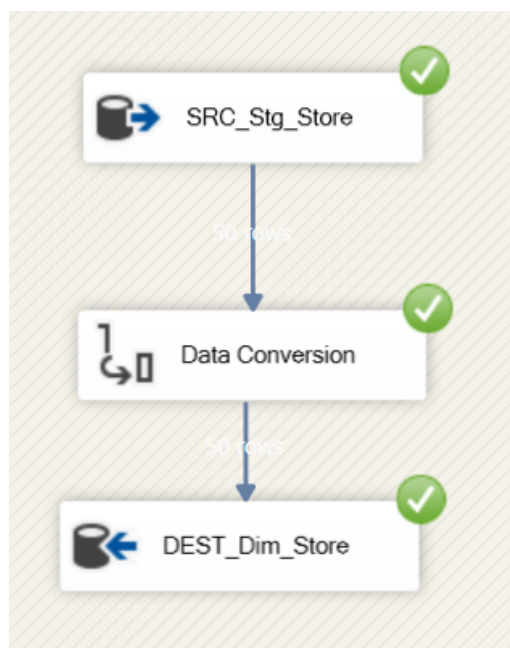


Figure 22:Data Flow of Dim_Store

3. Dim_Supplier

Dim_Supplier table store supplier related information and it used to evaluate supplier performance. This table implemented as SCD Type 2 to track changes in supplier performance over time.

- Data Source – product_supplier.csv
- Data Conversion – convert supplier_id to DT_WSTR
- SCD Transformation -track historical supplier changes
- Destination – Dim_Supplier

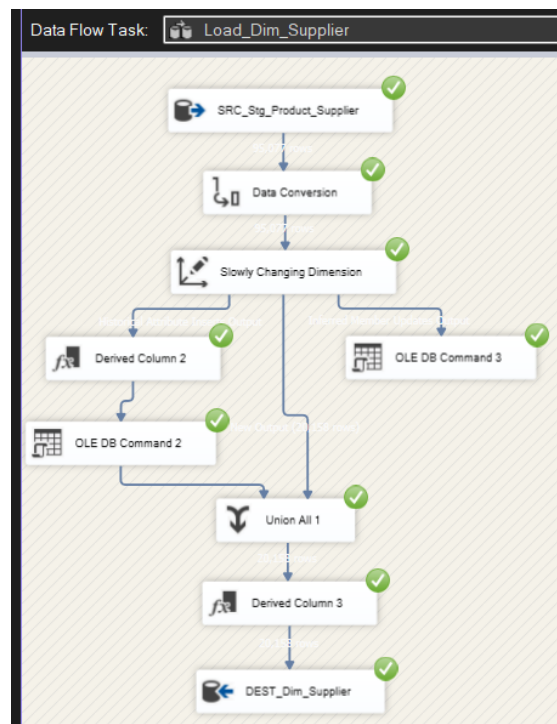


Figure 23:Data Flow of Dim_Supplier

4.Dim_Storage

Dim_Storage table captures storage conditions that affect product quality and spoilage.

- Data Source – storage_conditions.csv
- Destination – Dim_Storage

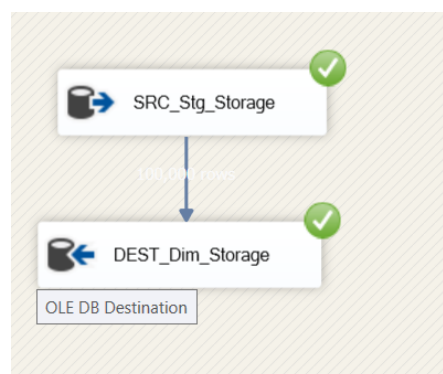


Figure 24:Data Flow for Dim_Storage

5. Dim_Date

Dim_Date table provides time_based attributes to support tren and time-series analysis.

- Data Source – date.csv
- Derived Columns – quarter, week_of_year
- Destination – Dim_Date

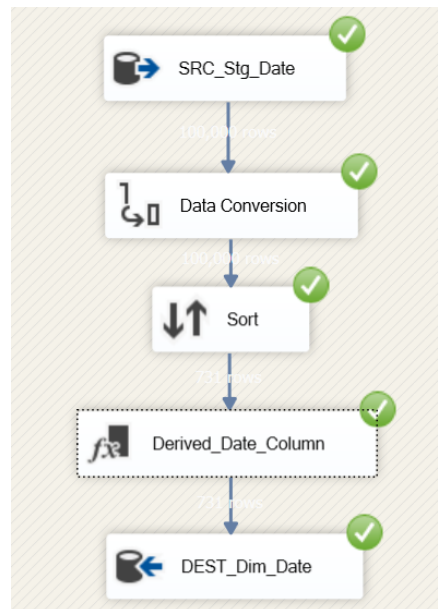


Figure 25: Data Flow of Dim_Date

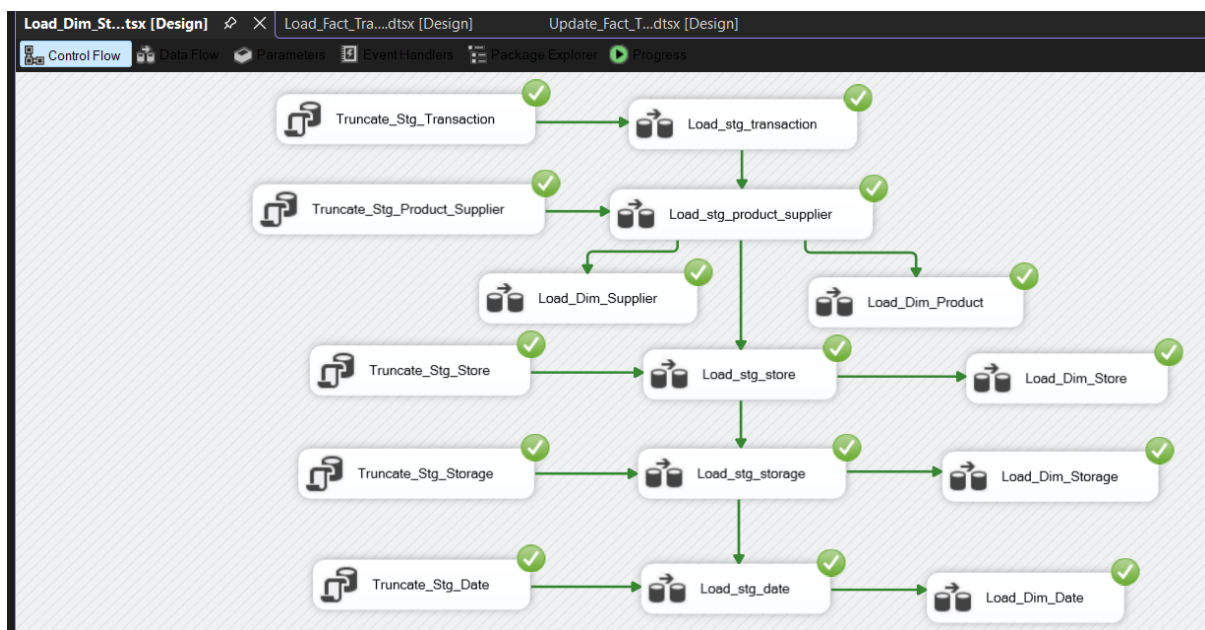


Figure 26: Control flow of Load_Dim package connecting staging and dimension loading sequence

Package 2: Load_Fact_Transaction.dtsx

Fact_Transaction table loads after all dimension tables are successfully mapped and populated,

Steps in package 2

1.Extract data from SQL Server table (dbo.TransactionData)

2.Apply transformations

- Data conversion
- Derived columns (price rounding, calculations)

3.Perform Lookup transformations - Map natural keys to surrogate keys (SK) from dimension tables

4.Handle errors

Implemented using SSIS error outputs. Unmatched records are redirected to a separate error table (Fact_Lookup_Error) during lookup transformation. A derived column is used to capture the source of the error, and this approach prevents ETL failure and follows correction of data quality issues.

- Lookup Transformation - Redirect failed rows to error output tables for debugging.
- Derived Column – add error_source column to indicate which lookup failed.
- Benefit – Prevents ETL failure and allow checking data quality

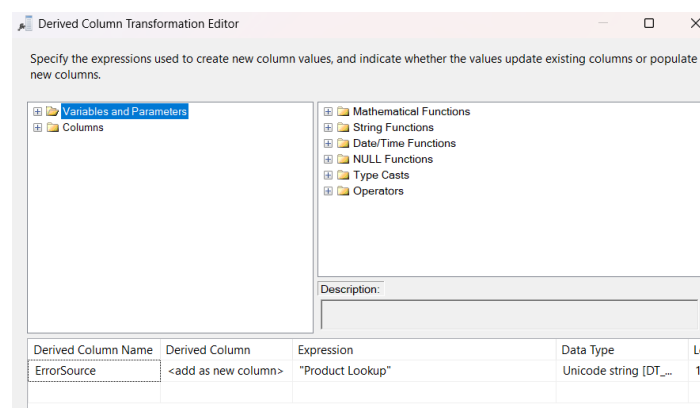


Figure 27: Derived Column (Error Source) for error handling

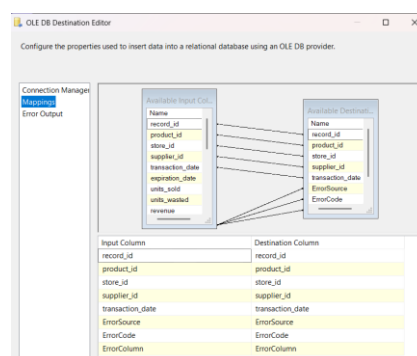


Figure 28: error handling for Lookup_Product

5. Load data into Fact_Transaction

- Source – dbo.TransactionData
- Data conversion - ensure all numeric and data fields are matched DW schema
- Lookup transformation – map natural keys to surrogate keys. Optimized using chaching to improve ETL performance.
 - Product→ product_key
 - Store→ store_key
 - Supplier→ supplier_key
 - Date→ date_key
 - Storage→ storage_key (no mapping between transaction and storage table, a default storage key (1) used to represent “unknown” storage to ensure integrity.)
- Derived Columns – add error source column for error handling
- Error Handling
- Destination – Fact_Transaction

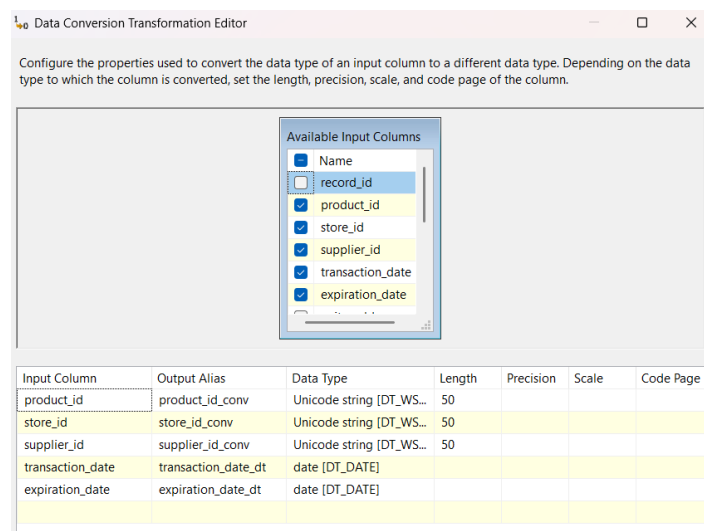


Figure 29: Data Conversion for Fact_Transaction

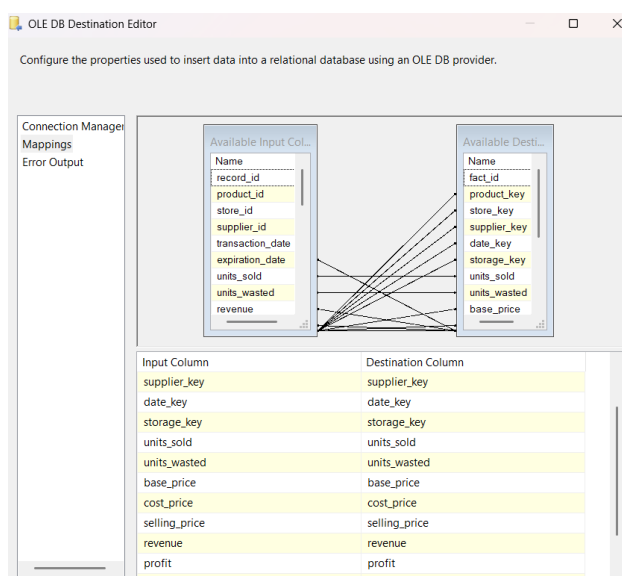


Figure 42: Fact_Transaction final mapping

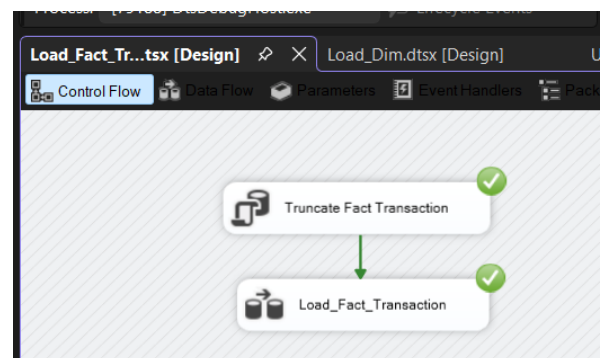


Figure 43: Control Flow of Fact_Transaction

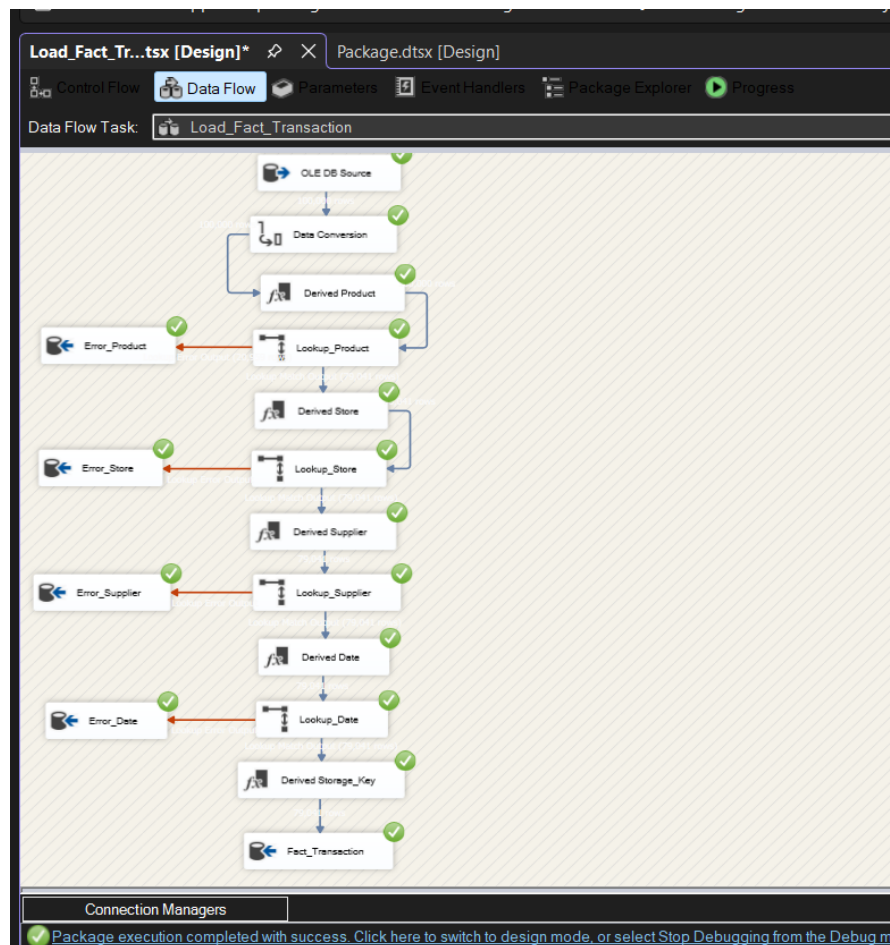


Figure 30: Data Flow of Fact_Transaction

5.4 Execution Order

1. Load_Dim_Product (SCD)
2. Load_Dim_Store
3. Load_Dim_Supplier (SCD)
4. Load_Dim_Storage
5. Load_Date
6. Load_Fact_Transaction

In ETL execution first load Dimensions and last Fact_Transaction. This ensures all dimension tables are populated before the fact table, maintaining referential integrity.

5.5 Validation of ETL results

1. Row count validation

```

77 SELECT COUNT(*) AS Product_Dimension FROM Dim_Product;
78 SELECT COUNT(*) AS Supplier_Dimension FROM Dim_Supplier;
79 SELECT COUNT(*) AS Store_Dimension FROM Dim_Store;
80 SELECT COUNT(*) AS Date_Dimension FROM Dim_Date;
81 SELECT COUNT(*) AS Storage_Dimension FROM Dim_Storage;
82 SELECT COUNT(*) AS Fact_Transaction FROM Fact_Transaction;
83

```

110 % No issues found

Results Messages

	Product_Dimension
1	61271

	Supplier_Dimension
1	19848

	Store_Dimension
1	50

	Date_Dimension
1	731

	Storage_Dimension
1	100000

	Fact_Transaction
1	100000

Query executed successfully.

Figure 31: Row count of Dim Tables

2. Sample Data Validation

```

170
171 SELECT TOP 10 * FROM Dim_Product;
172

```

110 % 31 0

Results Messages

	product_key	product_id	product_name	category	spoilage_sensitivity	start_date	end_date	is_current
1	1	BAK_DON_743	Donuts	Bakery	0.5	2026-03-26	NULL	1
2	2	MEA_SAU_338	Sausages	Meat	0.9	2026-03-26	NULL	1
3	3	BAK_BAG_799	Bagels	Bakery	0.5	2026-03-26	NULL	1
4	4	PHA_VAC_801	Vaccines	Pharmaceuticals	0.85	2026-03-26	NULL	1
5	5	REA_FRE_422	Fresh Pasta	Ready_to_Eat	0.9	2026-03-26	NULL	1
6	6	PHA_EYE_369	Eye Drops	Pharmaceuticals	0.85	2026-03-26	NULL	1
7	7	BEV_FRE_873	Fresh Juice	Beverages	0.4	2026-03-26	NULL	1
8	8	FRO_FRO_710	Frozen Dinners	Frozen_Meals	0.3	2026-03-26	NULL	0
9	9	PHA_ANT_887	Antibiotics	Pharmaceuticals	0.85	2026-03-26	NULL	1
10	10	BAK_CRO_880	Croissants	Bakery	0.5	2026-03-26	NULL	1

Figure 32: Dim_Product

```
174
175 SELECT TOP 10 * FROM Dim_Store;
176
```

110 % 31 0

Results Messages

	store_key	store_id	region
1	1	STORE_046	West
2	2	STORE_030	Southwest
3	3	STORE_035	Midwest
4	4	STORE_003	Midwest
5	5	STORE_042	West
6	6	STORE_026	West
7	7	STORE_041	Midwest
8	8	STORE_036	Northeast
9	9	STORE_019	Northeast
10	10	STORE_001	Northeast

Figure 33: Dim_Store

```
181
182 SELECT TOP 10 * FROM Dim_Storage;
183
```

110 % 31 0

Results Messages

	storage_key	storage_temp	temp_deviation	handling_score	packaging_score
1	1	21	1	9	10
2	2	0.7	0.3	8	8
3	3	21.3	1.3	4	8
4	4	8	3	5	9
5	5	3.5	0.5	5	6
6	6	4.5	0.5	10	8
7	7	8.2	3.2	8	9
8	8	-18.5	1.5	5	7
9	9	4.1	0.9	5	7
10	10	21.1	1.1	4	8

Figure 34: Dim_Storage

```
187
188 SELECT TOP 10 * FROM Dim_Supplier;
189
```

110 % 31 0

Results Messages

	supplier_key	supplier_id	supplier_score	start_date	end_date	is_current
1	1	SUPPLIER_03	9	2026-03-26	NULL	1
2	2	SUPPLIER_12	6	2026-03-26	NULL	1
3	3	SUPPLIER_08	9	2026-03-26	NULL	1
4	4	SUPPLIER_11	6	2026-03-26	NULL	1
5	5	SUPPLIER_15	6	2026-03-26	NULL	1
6	6	SUPPLIER_12	6	2026-03-26	NULL	1
7	7	SUPPLIER_06	8	2026-03-26	NULL	1
8	8	SUPPLIER_01	7	2026-03-26	NULL	1
9	9	SUPPLIER_14	6	2026-03-26	NULL	1
10	10	SUPPLIER_10	8	2026-03-26	NULL	1

Figure 35: Dim_Supplier

```
191 SELECT TOP 10 * FROM Dim_Date;
```

110 % 31 0

Results Messages

	date_key	full_date	day	month	year	day_of_week	is_weekend	quarter	week_of_year
1	20230101	2023-01-01	1	1	2023	1	1	1	1
2	20230102	2023-01-02	2	1	2023	2	0	1	1
3	20230103	2023-01-03	3	1	2023	3	0	1	1
4	20230104	2023-01-04	4	1	2023	4	0	1	1
5	20230105	2023-01-05	5	1	2023	5	0	1	1
6	20230106	2023-01-06	6	1	2023	6	0	1	1
7	20230107	2023-01-07	7	1	2023	7	1	1	1
8	20230108	2023-01-08	8	1	2023	1	1	1	2
9	20230109	2023-01-09	9	1	2023	2	0	1	2
10	20230110	2023-01-10	10	1	2023	3	0	1	2

Figure 36: Dlm_Date

```
195 SELECT TOP 10 * FROM Fact_Transaction;
```

110 % 31 0

Results Messages

	fact_id	product_key	store_key	supplier_key	date_key	storage_key	units_sold	units_wasted	base_price	cost_price	selling_price	revenue	profit	waste_cost	expiration_date
1	1	1	1	1	20240925	1	138	20	2.6	1.22	2.6	358.8	166.04	24.4	2024-09-29
2	2	2	2	2	20230414	1	251	102	8.25	4.18	8.25	2070.75	595.21	426.36	2023-04-21
3	3	3	3	3	20241025	1	483	0	4.12	2.12	1.28	618.24	-405.72	0	2024-10-27
4	4	4	4	4	20231129	1	477	0	209.56	87.99	209.56	99960.12	57988.89	0	2024-02-17
5	5	5	5	5	20230806	1	391	0	6.86	3.03	5.28	2064.48	879.75	0	2023-08-09
6	6	6	6	2	20240821	1	0	270	121.37	84.87	121.37	0	-22914.9	22914.9	2025-07-06
7	7	7	7	7	20230218	1	41	1	8.34	4.39	8.34	341.94	157.56	4.39	2023-02-27
8	8	8	8	8	20240424	1	0	463	14.88	6.46	14.88	0	-2990.98	2990.98	2025-02-16
9	9	9	9	9	20241018	1	379	0	91.71	36.77	91.71	34758.09	20822.26	0	2025-10-17
10	10	10	7	10	20230702	1	111	0	12.94	7.15	12.94	1436.34	642.69	0	2023-07-08

Figure 37: Fact_Transaction

3.Referential Integrity Check – ensure all foreign keys in fact table match dimensions tables.

```
167 SELECT f.product_key
168 FROM Fact_Transaction f
169 LEFT JOIN Dim_Product d
170 ON f.product_key = d.product_key
171 WHERE d.product_key IS NULL;
```

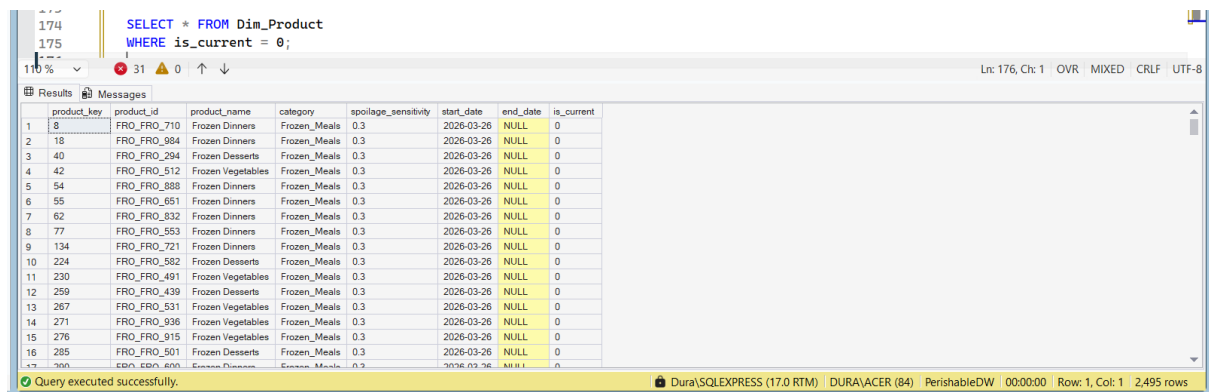
10 % 31 0

Results Messages

	product_key
--	-------------

Figure 38: Integrity Check

4.SCD Validation for Dim_Product – ensure historical records are maintained.



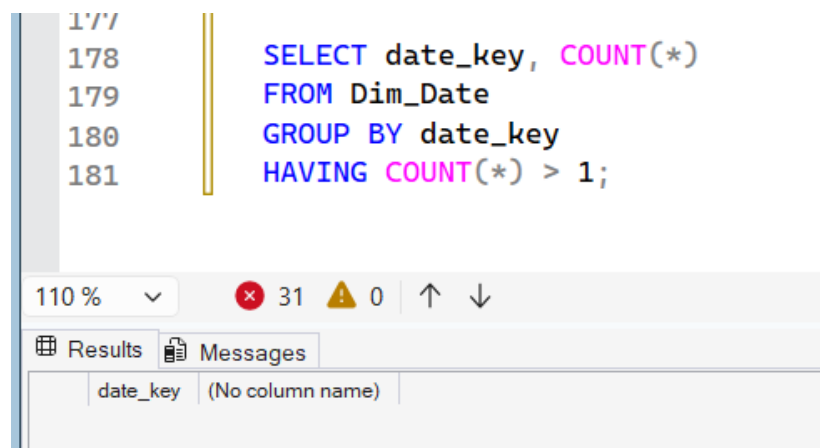
```

SELECT * FROM Dim_Product
WHERE is_current = 0;
  
```

product_key	product_id	product_name	category	spoilage_sensitivity	start_date	end_date	is_current
8	FRO_FRO_710	Frozen Dinners	Frozen_Meals	0.3	2026-03-26	NULL	0
18	FRO_FRO_984	Frozen Dinners	Frozen_Meals	0.3	2026-03-26	NULL	0
40	FRO_FRO_294	Frozen Desserts	Frozen_Meals	0.3	2026-03-26	NULL	0
42	FRO_FRO_512	Frozen Vegetables	Frozen_Meals	0.3	2026-03-26	NULL	0
54	FRO_FRO_808	Frozen Dinners	Frozen_Meals	0.3	2026-03-26	NULL	0
55	FRO_FRO_651	Frozen Dinners	Frozen_Meals	0.3	2026-03-26	NULL	0
62	FRO_FRO_832	Frozen Dinners	Frozen_Meals	0.3	2026-03-26	NULL	0
77	FRO_FRO_553	Frozen Dinners	Frozen_Meals	0.3	2026-03-26	NULL	0
134	FRO_FRO_721	Frozen Dinners	Frozen_Meals	0.3	2026-03-26	NULL	0
224	FRO_FRO_582	Frozen Desserts	Frozen_Meals	0.3	2026-03-26	NULL	0
230	FRO_FRO_491	Frozen Vegetables	Frozen_Meals	0.3	2026-03-26	NULL	0
259	FRO_FRO_439	Frozen Desserts	Frozen_Meals	0.3	2026-03-26	NULL	0
267	FRO_FRO_531	Frozen Vegetables	Frozen_Meals	0.3	2026-03-26	NULL	0
271	FRO_FRO_936	Frozen Vegetables	Frozen_Meals	0.3	2026-03-26	NULL	0
276	FRO_FRO_915	Frozen Vegetables	Frozen_Meals	0.3	2026-03-26	NULL	0
285	FRO_FRO_501	Frozen Desserts	Frozen_Meals	0.3	2026-03-26	NULL	0
300	FRO_FRO_600	Frozen Dinners	Frozen_Meals	0.3	2026-03-26	NULL	0

Figure 39: SCD Validation of Dim_Product

5.Duplicate Check for Dim_Date



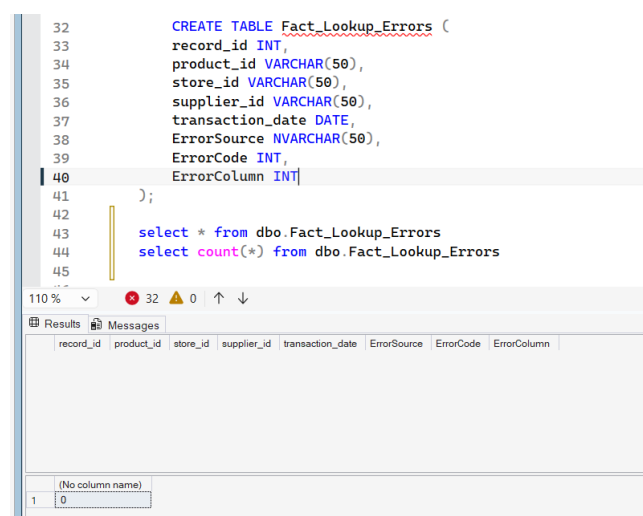
```

SELECT date_key, COUNT(*)
FROM Dim_Date
GROUP BY date_key
HAVING COUNT(*) > 1;
  
```

date_key	(No column name)
----------	------------------

Figure 40: Duplicate records check for Dim_Date

6.Error Handling Validation (Fact_Lookup_Errors)



```

CREATE TABLE Fact_Lookup_Errors (
record_id INT,
product_id VARCHAR(50),
store_id VARCHAR(50),
supplier_id VARCHAR(50),
transaction_date DATE,
ErrorSource NVARCHAR(50),
ErrorCode INT,
ErrorColumn INT)
  
```

```

select * from dbo.Fact_Lookup_Errors
select count(*) from dbo.Fact_Lookup_Errors
  
```

record_id	product_id	store_id	supplier_id	transaction_date	ErrorSource	ErrorCode	ErrorColumn
1							

Figure 41: Error Handling

Task 6: ETL development – Accumulating fact tables

Accumulating fact table approach was implemented to track transaction lifecycle updates by updating the same record over time to reflect the progress of a single process. Accumulating fact tables allow updates to capture transaction creation time, completion time and processing duration to enable analysis of process duration and operational efficiency. In the perishable goods domain, tracking how long a transaction takes to complete is important for identifying delays that makes leads to spoilage or revenue loss and supports performance analysis across stores, suppliers and products.

1. Modify Existing Fact Transaction

- accm_txn_create_time - store initial load timestamp when a transaction is first created.
- accm_txn_complete_time – store when transaction is completed and update later via ETL process.
- txn_process_time_hours – calculate total processing time in hours between creation and completion to enable performance and delay analysis.

Transaction Lifecycle in DW

1. A new transaction is inserted into the fact table.
2. accm_txn_create_time is populated with current system timestamp
3. accm_txn_complete_time is initially NULL
4. When completion date arrives, the same record is updated.
5. txn_process_time_hours is calculated dynamically.

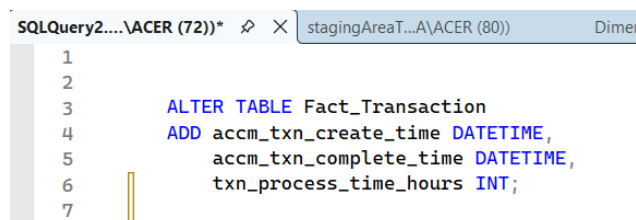


Figure 42: Update Fact Transaction Columns

2. Add derived column for accm_txn_create_time to get the date

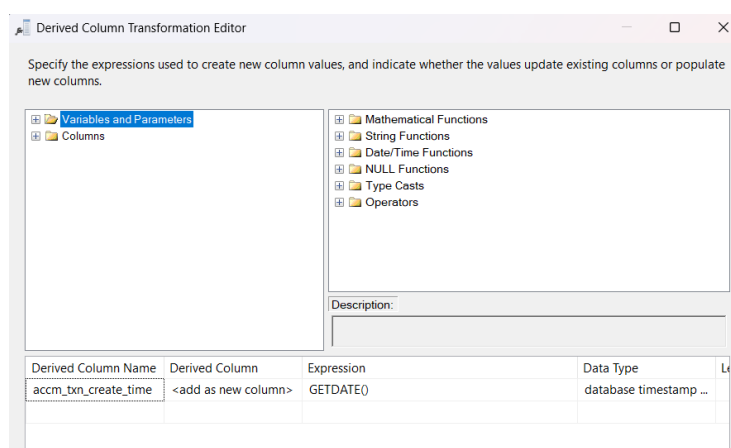


Figure 43: Derived date for accm_txn_create_time

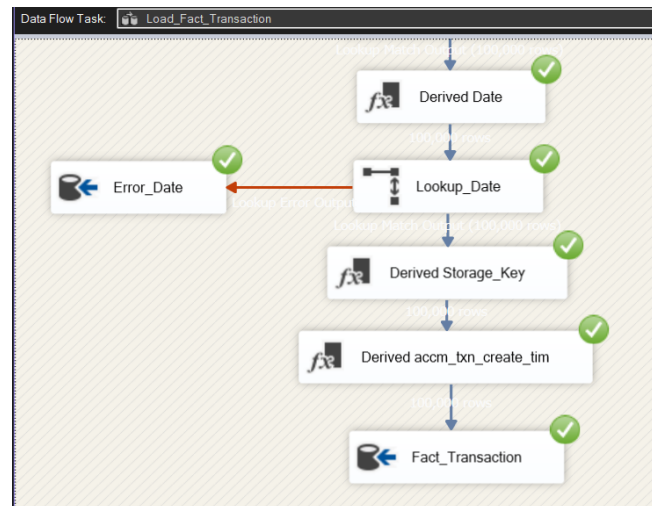


Figure 44: Added derived column of accm_txn_create_time

3.Initial Load Logic

- accm_txn_create_time was set using the system date
- accm_txn_complete_time initially it is left NULL

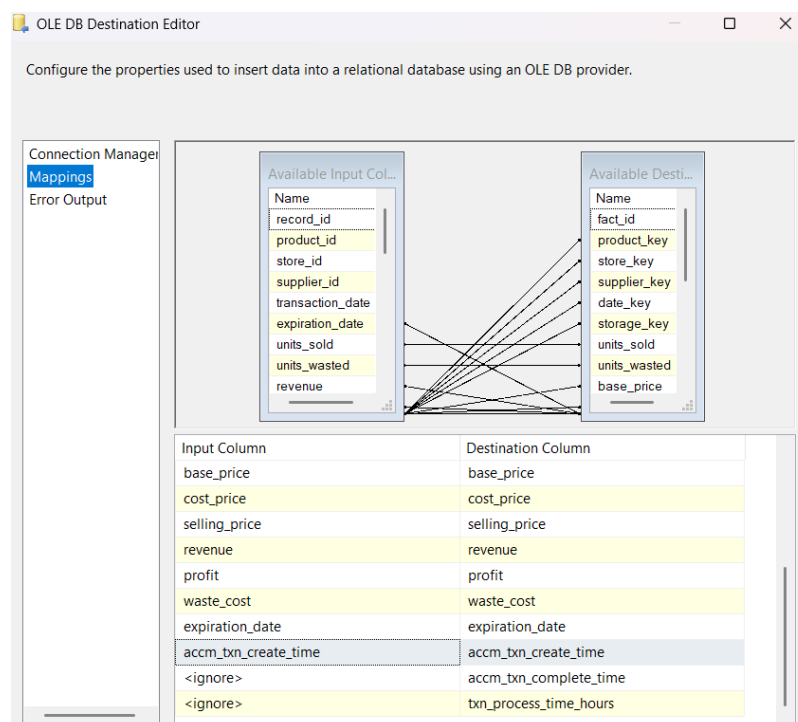


Figure 45: Fact_Transaction mapping with new columns

4. Create Update Dataset (txn_updates.txt)

```
txn_id,accm_txn_complete_time
1,2026-03-30 10:00:00
2,2026-03-30 12:00:00
3,2026-03-30 15:00:00
4,2026-03-30 18:00:00
5,2026-03-30 20:00:00
6,2026-03-30 09:30:00
7,2026-03-30 11:15:00
8,2026-03-30 13:45:00
9,2026-03-30 16:20:00
10,2026-03-31 19:10:00
11,2026-03-31 08:00:00
12,2026-03-31 10:30:00
13,2026-03-31 12:45:00
14,2026-03-31 14:10:00
15,2026-03-31 17:25:00
16,2026-03-31 09:50:00
17,2026-03-31 11:40:00
18,2026-03-31 13:35:00
19,2026-03-31 15:55:00
20,2026-03-31 18:05:00
```

Figure 46: Create new source file for update time

5. Update ETL Package (Update_Fact_Transaction)

Update processes use the transaction natural key (txn_id) to locate existing records in the fact table. The OLE DB Command transformation performs row by row update to populate the completion timestamp and recalculate processing time which suitable for small, moderate datasets where immediate updates are required. It is appropriate for demonstrate incremental updates in SSIS.

- Flat file source – txn_updates.csv
- Data Conversion – convert datetime fields
- Derived Column – calculate processing time
- Process Time Calculation - DATEDIFF("hour", accm_txn_create_time, accm_txn_complete_time)
- OLE DB Command – update fact table

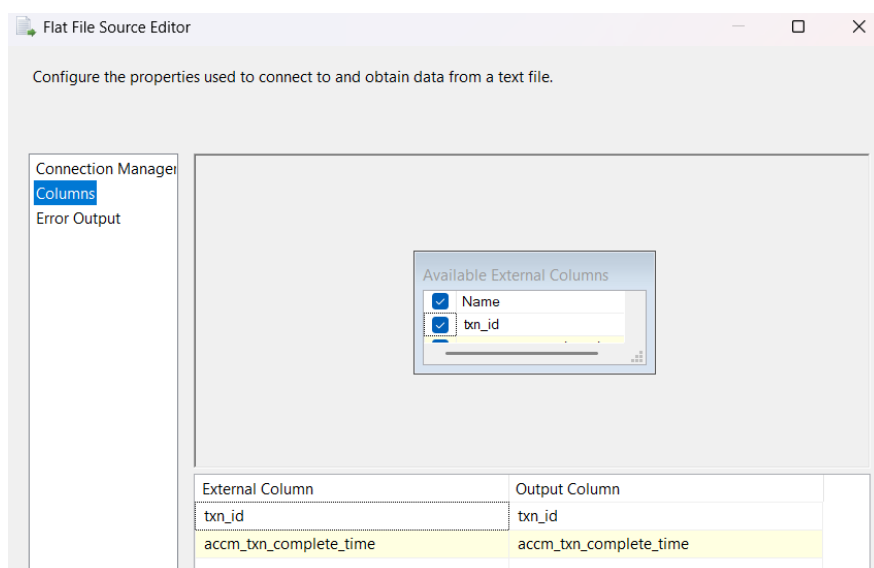


Figure 47: Flat File source configure – txn_update.csv

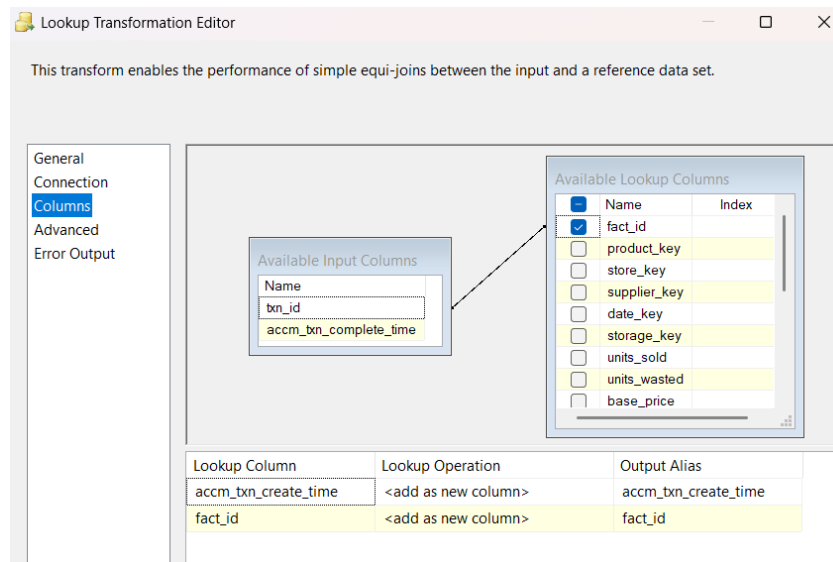


Figure 48: Lookup for Fact_Transaction to match txn_id

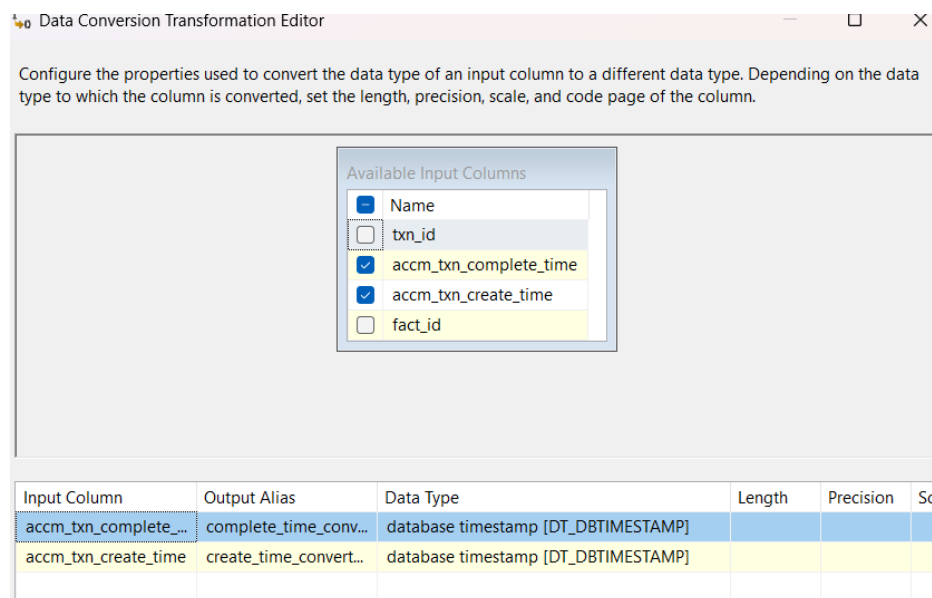


Figure 49: Data Conversion of complete time and create time to DB timestamp

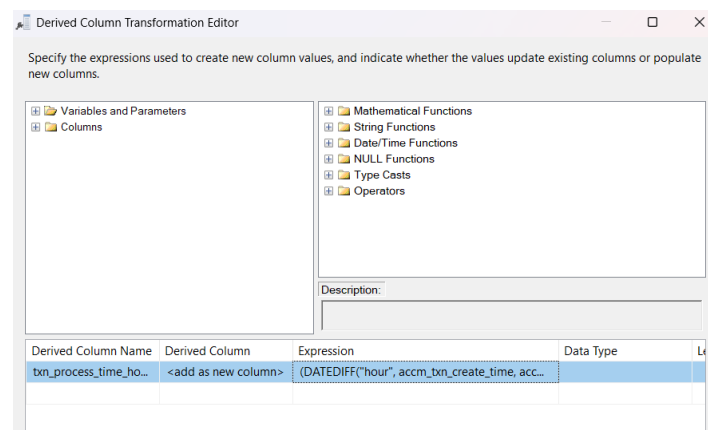


Figure 50: Derived column for process_time_hours

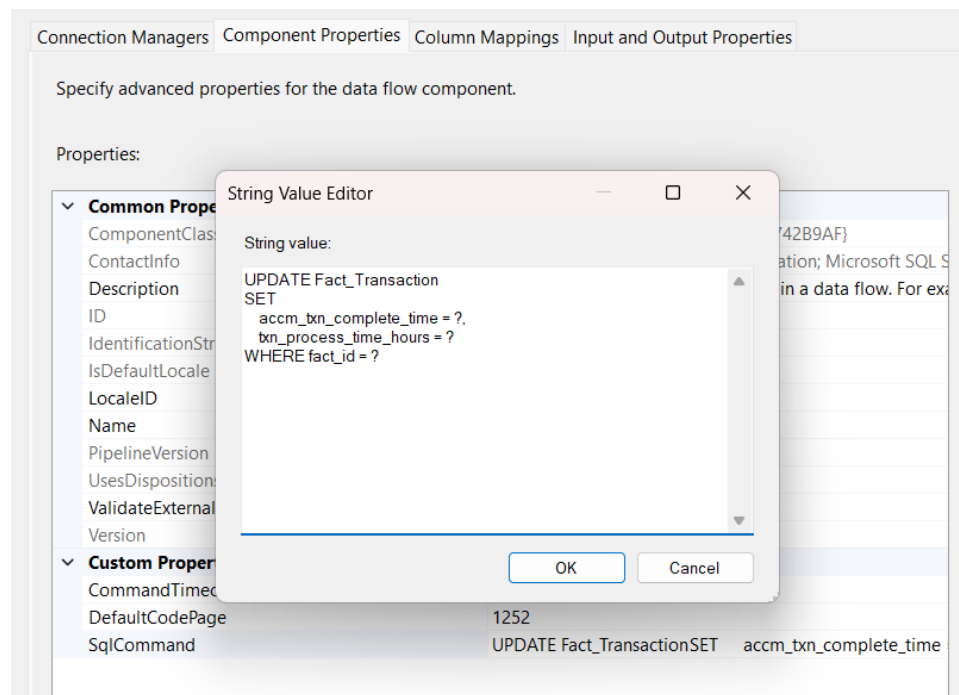


Figure 51: OLE DB Command

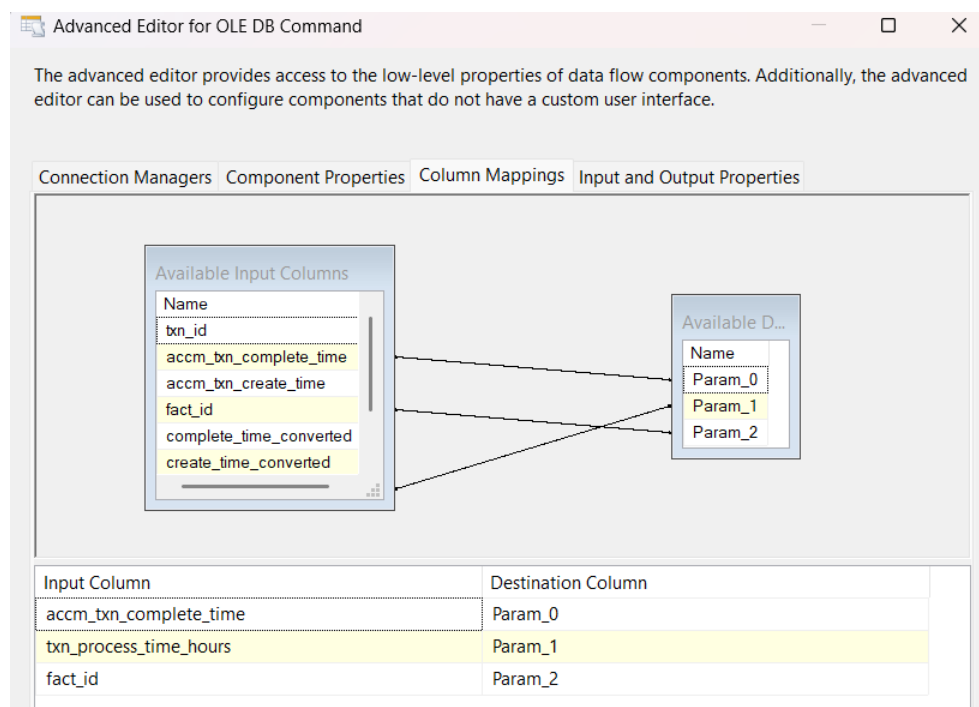


Figure 52: Mapping to Fact_Transaction table

6.Error Handling

- Configured Lookup to redirect unmatched rows
- Errors stored in Fact_Lookup_Errors Table

```

44 ALTER TABLE Fact_Lookup_Errors
45 ADD txn_id INT
46
47 ALTER TABLE Fact_Lookup_Errors
48 ADD accm_txn_complete_time DATETIME
49

```

```

20 SELECT COUNT(*) FROM Fact_Lookup_Errors;
21 select * from Fact_Lookup_Errors
22
23

```

10 % ✓ No issues found

Results Messages

(No column name)
0

record_id	product_id	store_id	supplier_id	transaction_date	ErrorSource	ErrorCode	ErrorColumn	txn_id	accm_txn_complete_time
-----------	------------	----------	-------------	------------------	-------------	-----------	-------------	--------	------------------------

Figure 53: Add new columns to Fact_Lookup_Errors Table

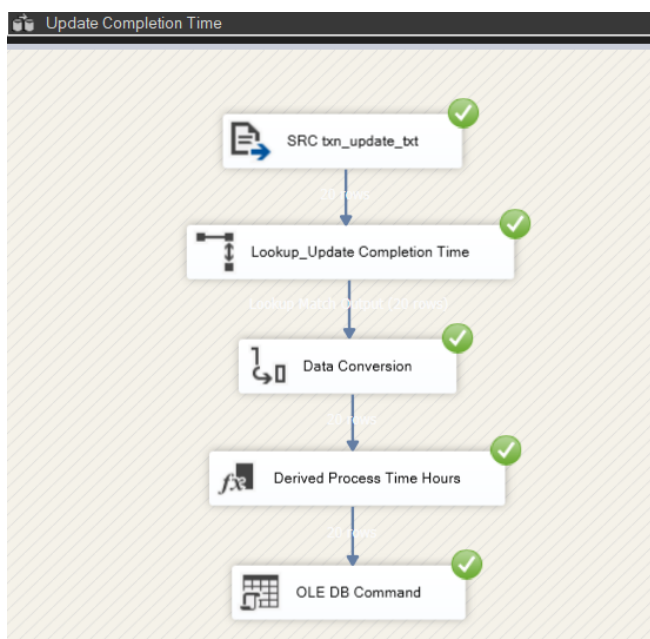


Figure 68: Data Flow of Update_Complete_Time

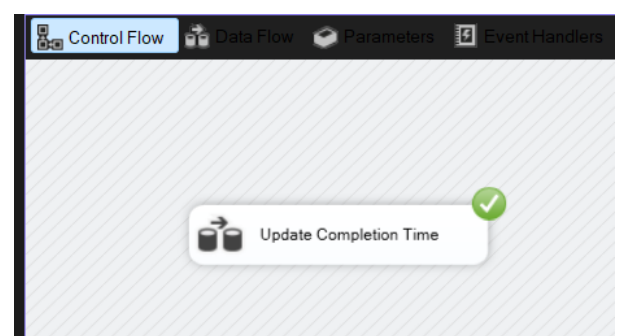


Figure 69: Control Flow of Update Completion Time

7.Validation Queries

```

95      SELECT
96          fact_id,
97          accm_txn_create_time,
98          accm_txn_complete_time,
99          txn_process_time_hours
100     FROM Fact_Transaction
101     WHERE accm_txn_complete_time IS NOT NULL;
  
```

110 % 4 0

Results Messages

	fact_id	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
2	2	2026-03-28 12:28:03.913	2026-03-30 12:00:00.000	48
3	3	2026-03-28 12:28:03.913	2026-03-30 15:00:00.000	51
4	4	2026-03-28 12:28:03.913	2026-03-30 18:00:00.000	54
5	5	2026-03-28 12:28:03.913	2026-03-30 20:00:00.000	56
6	6	2026-03-28 12:28:03.913	2026-03-30 09:30:00.000	45
7	7	2026-03-28 12:28:03.913	2026-03-30 11:15:00.000	47
8	8	2026-03-28 12:28:03.913	2026-03-30 13:45:00.000	49
9	9	2026-03-28 12:28:03.913	2026-03-30 16:20:00.000	52
10	10	2026-03-28 12:28:03.913	2026-03-31 19:10:00.000	79
11	11	2026-03-28 12:28:03.913	2026-03-31 08:00:00.000	68
12	12	2026-03-28 12:28:03.913	2026-03-31 10:30:00.000	70
13	13	2026-03-28 12:28:03.913	2026-03-31 12:45:00.000	72
14	14	2026-03-28 12:28:03.913	2026-03-31 14:10:00.000	74
15	15	2026-03-28 12:28:03.913	2026-03-31 17:25:00.000	77
16	16	2026-03-28 12:28:03.913	2026-03-31 09:50:00.000	69
17	17	2026-03-28 12:28:03.913	2026-03-31 11:40:00.000	71
18	18	2026-03-28 12:28:03.913	2026-03-31 13:35:00.000	73
19	19	2026-03-28 12:28:03.913	2026-03-31 15:55:00.000	75
20	20	2026-03-28 12:28:03.913	2026-03-31 18:05:00.000	78

Query executed successfully.

Figure 54: Validation of accumulative tables

```

38      SELECT COUNT(*) AS updated_rows
39     FROM Fact_Transaction
40     WHERE accm_txn_complete_time IS NOT NULL;
  
```

110 % No issues found

Results Messages

	updated_rows
1	20

Figure 55: Count rows

```

104     SELECT TOP 10 *
105     FROM Fact_Transaction
106     ORDER BY accm_txn_complete_time DESC;
  
```

110 % 5 0

Ln: 102, Ch: 1 SPC CRLF UTF-8

Results Messages

	fact_id	product_key	store_key	supplier_key	date_key	storage_key	units_sold	units_wasted	base_price	cost_price	selling_price	revenue	profit	waste_cost	expiration_date	accm_txn_create_time	accm_txn_complete_time	txn_process
1	10	10	7	10	20230702	1	111	0	12.94	7.15	12.94	1436.34	642.69	0	2023-07-08	2026-03-28 12:28:03.913	2026-03-31 19:10:00.000	79
2	20	20	11	3	20241029	1	119	0	95.66	52.2	95.66	11383.54	5171.74	0	2026-06-01	2026-03-28 12:28:03.913	2026-03-31 18:05:00.000	78
3	15	15	14	3	20231213	1	40	0	1.05	0.64	0.85	34	8.4	0	2023-12-16	2026-03-28 12:28:03.913	2026-03-31 17:25:00.000	77
4	19	19	15	5	20230524	1	236	0	2.75	1.86	2.75	649	210.04	0	2023-06-08	2026-03-28 12:28:03.913	2026-03-31 15:55:00.000	75
5	14	14	13	2	20230725	1	0	71	227.48	126.31	227.48	0	-8968.01	8968.01	2024-05-18	2026-03-28 12:28:03.913	2026-03-31 14:10:00.000	74
6	18	18	6	4	20230225	1	103	5	16.81	7.28	16.81	1731.43	945.19	36.4	2023-06-12	2026-03-28 12:28:03.913	2026-03-31 13:35:00.000	73
7	13	13	12	13	20241106	1	118	0	270.92	157.67	211.32	24935.76	6330.7	0	2025-11-17	2026-03-28 12:28:03.913	2026-03-31 12:45:00.000	72
8	17	17	16	17	20230906	1	226	0	6.26	3.31	2	452	-296.06	0	2023-09-09	2026-03-28 12:28:03.913	2026-03-31 11:40:00.000	71
9	12	12	11	1	20230301	1	279	120	14.25	9.19	14.25	3975.75	308.94	1102.8	2023-03-05	2026-03-28 12:28:03.913	2026-03-31 10:30:00.000	70
10	16	16	15	16	20231205	1	488	0	2.5	1.1	2.5	1220	683.2	0	2023-12-10	2026-03-28 12:28:03.913	2026-03-31 09:50:00.000	69

Figure 56: Query for descending complete time

109 | SELECT * FROM Fact_Transaction

110 % | 5 | 0 | ↑ | ↓ | Ln: 104, Ch: 16 | SPC | CRLF | UTF-8

	fact_id	product_key	store_key	supplier_key	date_key	storage_key	units_sold	units_wasted	base_price	cost_price	selling_price	revenue	profit	waste_cost	expiration_date	accm_txn_create_time	accm_txn_complete_time	txn_process_time
1	1	1	1	1	20240925	1	138	20	2.6	1.22	2.6	358.8	166.04	24.4	2024-09-29	2026-03-28 12:28:03.913	2026-03-30 10:00:00.000	46
2	2	2	2	2	20230414	1	251	102	8.25	4.18	8.25	2070.75	595.21	426.36	2023-04-21	2026-03-28 12:28:03.913	2026-03-30 12:00:00.000	48
3	3	3	3	3	20241025	1	483	0	4.12	2.12	1.28	618.24	-405.72	0	2024-10-27	2026-03-28 12:28:03.913	2026-03-30 15:00:00.000	51
4	4	4	4	4	20231129	1	477	0	209.56	87.99	209.56	99960.12	57988.89	0	2024-02-17	2026-03-28 12:28:03.913	2026-03-30 18:00:00.000	54
5	5	5	5	5	20230806	1	391	0	6.86	3.03	5.26	2064.48	879.75	0	2023-08-09	2026-03-28 12:28:03.913	2026-03-30 20:00:00.000	56
6	6	6	2	2	20240821	1	0	270	121.37	84.57	121.37	0	-22914.9	22914.9	2025-07-06	2026-03-28 12:28:03.913	2026-03-30 09:30:00.000	45
7	7	7	7	7	20230218	1	41	1	9.34	4.39	9.34	341.94	157.56	4.39	2023-02-27	2026-03-28 12:28:03.913	2026-03-30 11:15:00.000	47
8	8	8	8	8	20240424	1	0	463	14.88	6.46	14.88	0	-2990.98	2990.98	2025-02-16	2026-03-28 12:28:03.913	2026-03-30 13:45:00.000	49
9	9	9	9	9	20241018	1	379	0	91.71	36.77	91.71	34758.09	20822.26	0	2025-10-17	2026-03-28 12:28:03.913	2026-03-30 16:20:00.000	52
10	10	10	7	10	20230702	1	111	0	12.94	7.15	12.94	1436.34	642.69	0	2023-07-08	2026-03-28 12:28:03.913	2026-03-31 19:10:00.000	79

Figure 57: Final Fact Transaction Table Data

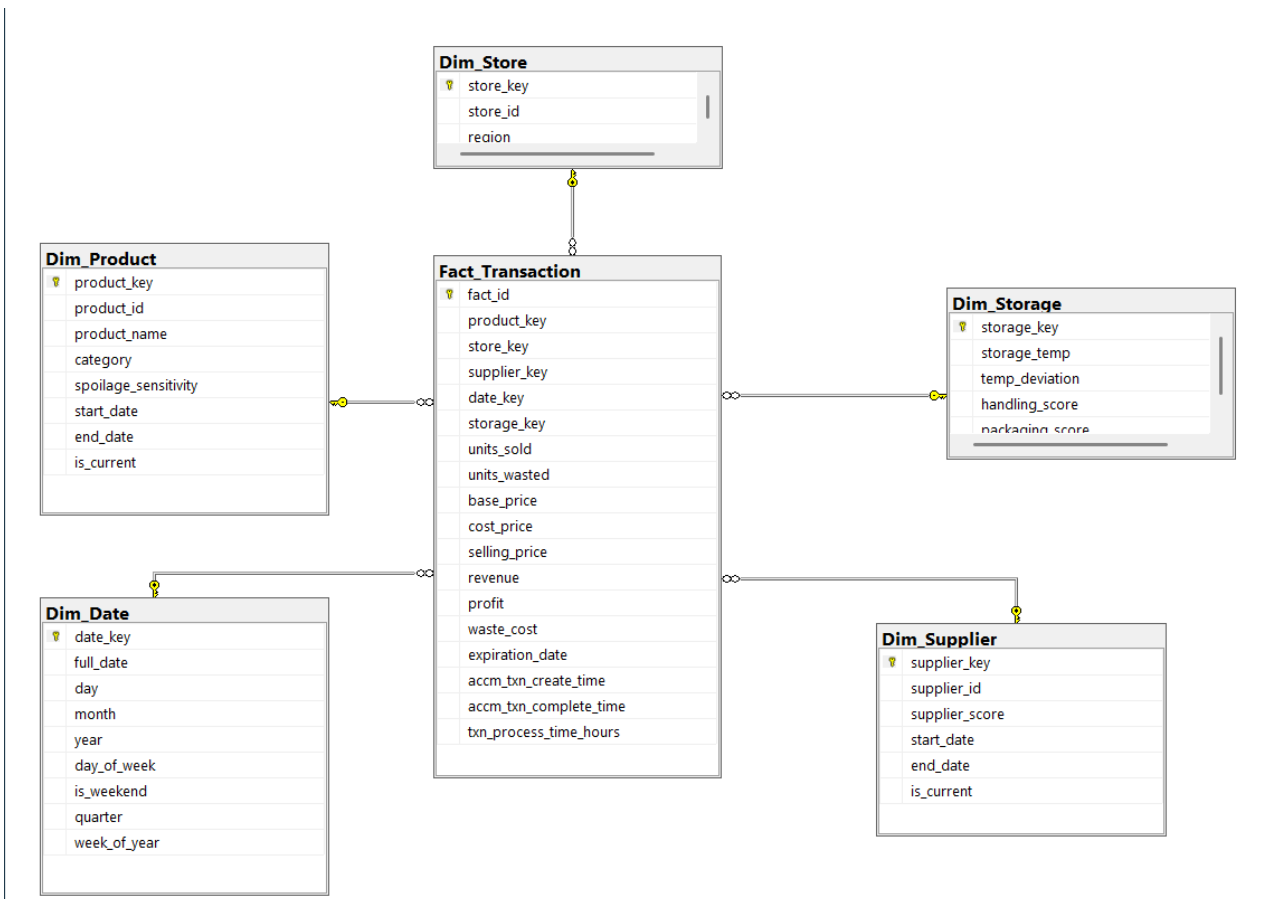


Figure 58: Final Star Schema - Relational Diagram